From Prediction to Persuasion: Agentic Recommendation Reason Generation for Regulatory-Compliant Financial AI

Seonkyu Jeong[1], Euncheol Sim[1], Youngchan Kim[1]

[1]*Independent Research*
Corresponding author: Seonkyu Jeong (ORCID: 0009-0005-3291-9112)

Abstract. Financial product recommendation systems must explain *why* a product is recommended — first and foremost because persuasion requires narrative, not probability: a customer who asks "Why was this card recommended to me?" needs a reason they can accept, not a score they must trust. Regulators (Korean FSS, EU AI Act) are one important audience for such explanations, but the primary driver is the human act of persuasion itself. We present a multi-stage pipeline that bridges the gap between model prediction and human persuasion: (1) adaptive knowledge distillation from a heterogeneous-expert PLE teacher to per-task LGBM students, with three-layer fallback (distill / direct hard-label / rule-based) and teacher threshold gating that ensures service continuity when teacher quality varies across tasks, enabling CPU-only inference; (2) a multi-agent recommendation reason generation pipeline where three specialized serving agents (Feature Selector, Reason Generator, Safety Gate) collaboratively produce natural-language explanations grounded in business-mapped feature attributions; (3) two operational agents (OpsAgent and AuditAgent) that interpret monitoring outputs and compliance reports in natural language, enabling regulation-compliant MLOps for small teams without dedicated MLOps staff; (4) regulatory compliance by design, with built-in drift monitoring, fairness auditing, and governance reporting aligned to Korean FSS guidelines, the EU AI Act, and the Korean AI Basic Act; (5) a **per-prediction causal audit surface** that pairs explanation (Causal Explainability Head attribution) with reliability (Causal Guardrail coherence score) in a shared HMAC-signed hash-chained audit log, producing a regulator-usable per-decision record that satisfies GDPR Art. 22 meaningful-explanation and EU AI Act Art. 13 transparency requirements — the companion loss-dynamics paper produces the attribution vector (its causal-attribution finding) and the coherence score (its causal-guardrail findings); this paper routes both into the audit infrastructure. We evaluate distillation quality (v13 baseline: AUC gap < 3.6 percentage points across 7 binary tasks, mean 2.6 pp; the v14 re-run after the fidelity-gate fixes analysed in the Discussion (Section 8) reduces the mean gap to 0.58 pp), reason generation quality via automated compliance validation (L1 template coverage 100%, 13/13 tasks; compliance rules applied: suitability, consent, opt-out, profiling, disclosure), and Safety Gate reliability (5 PII patterns, 5 validation categories). The system targets low-risk products (check cards, deposits); investment and insurance recommendations are excluded from the deployment scope. The system achieves 120ms warm latency on AWS Lambda (L1 predict + 13 tasks) at a fraction of the cost of dedicated GPU inference servers, with cold start 6s (S3 model download) and L2a cache hit at 6ms (DynamoDB).

Keywords: Recommendation reason generation, Knowledge distillation, LLM agents, Regulatory compliance, Financial AI, EU AI Act

1 Introduction

1.1 The Persuasion Problem

The final deliverable of a financial recommendation system is not a probability but a *reason the customer can accept*. A model outputting $P(\text{acquire} \mid x) = 0.73$ provides no value to:

- The **customer** who asks "Why was this card recommended to me?"
- The **relationship manager** who needs a talking point for the sales call.
- The **regulator** who demands "Why was this decision made?" under Article 13 of the EU AI Act.

Existing explanation approaches are insufficient:

- **SHAP/LIME**: "feature_237 contributed 0.12" — no business meaning, unstable under perturbation [12, 15].
- **Template-based**: "Customers like you also bought X" — rigid, ignores individual context.
- **Direct LLM generation**: Hallucination risk, no grounding in actual model reasoning, regulatory non-compliance.

1.2 Our Approach

We propose a full-chain solution from prediction to persuasion:

1. **Knowledge Distillation**: A heterogeneous-expert PLE teacher (13 tasks, 7 experts, 1211 features — 1210 after the v14 preprocessing revision; see companion paper for architecture and ablation) is distilled into per-task LGBM students via adaptive threshold gating that routes each task to DISTILL (soft labels), DIRECT (hard labels), or SKIP (rule engine) based on teacher quality assessment. This enables CPU-only serving while a three-layer fallback guarantees service continuity under any model failure scenario.
2. **Multi-Agent Reason Generation**: Three specialized LLM agents collaborate in a pipeline — Feature Selector chooses explanation-worthy features, Reason Generator produces natural-language narratives, and Safety Gate validates regulatory compliance.
3. **Compliance by Design**: Drift monitoring, fairness auditing, audit trails, and governance reporting are embedded in the pipeline, not bolted on after deployment.

1.3 Contributions

1. **Adaptive Distillation with Three-Layer Fallback**: Teacher threshold gating automatically routes each of the 13 tasks to DISTILL (soft labels, 7 tasks), DIRECT (hard labels, 3 tasks), or SKIP (rule engine, 3 tasks) based on teacher quality relative to a 2x random baseline, ensuring service continuity per SR 11-7 model risk management requirements. Feature selection uses LGBM gain importance, consistent with the serving model perspective.
2. **Feature Business Reverse-Mapping**: A systematic interpretation registry that maps every feature to a business-interpretable description, enabling grounded explanation generation.
3. **3-Agent Reason Generation Pipeline**: Role-separated agents (selection → generation → safety) with independent improvement and audit logging.
4. **Safety Gate for Financial Compliance**: Automated checking for hallucination, inappropriate investment advice, and regulatory violations (Korean Financial Consumer Protection Act (금소법, hereafter KFCPA), Suitability Principle).
5. **5-Agent Architecture (3 Serving + 2 Ops)**: Beyond the 3 serving agents, two operational agents (OpsAgent, AuditAgent) interpret monitoring and compliance outputs in natural language, enabling small-team MLOps without dedicated MLOps staff.
6. **Regulatory Compliance Architecture**: Explicit mapping of system components to Korean FSS guidelines, EU AI Act articles, and the Korean AI Basic Act.
7. **Per-Prediction Causal Audit Pair**: An HMAC-signed, hash-chained audit record per decision that pairs **what** the model recommended (CEH attribution: top-K influential features plus full-vector SHA256) with **whether** it should be trusted (CG coherence score and threshold). Both flow through the same `AuditLogger` infrastructure (`log_attribution`, `log_guardrail`), and the hash-chain verifier detects tampering or selective deletion of either record class. Produces the concrete per-decision audit evidence that Art. 13 transparency and GDPR Art. 22 meaningful-explanation call for.
8. **Human Evaluation Protocol**: A systematic protocol designed for post-deployment expert evaluation, with automated compliance validation reported in this paper as an interim measure.

2 Related Work

2.1 Knowledge Distillation for Recommendation

Hinton et al. [8] established the teacher-student paradigm via temperature-scaled soft labels. In recommendation, distillation has been applied to compress deep models into lightweight servers. CKD (Collaborative Knowledge Distillation) and ranking distillation preserve recommendation-specific structure.

Gap: No prior work applies adaptive threshold gating to route distillation tasks based on teacher quality assessment, nor provides a structured three-layer fallback that guarantees service continuity under heterogeneous teacher performance across tasks.

2.2 Recommendation Explanation Generation

Recommendation explanation approaches fall into three categories:

Template-based methods map model outputs to prewritten phrases ("Recommended because you recently viewed similar products"). Amazon and Netflix use variants of this approach at scale. Templates are safe and fast but lack personalization and flexibility — the same template serves millions of different customer contexts.

Attention-based methods use model-internal attention weights as feature attributions for explanation. While more dynamic than templates, attention weights are contested as reliable explanations, and the resulting

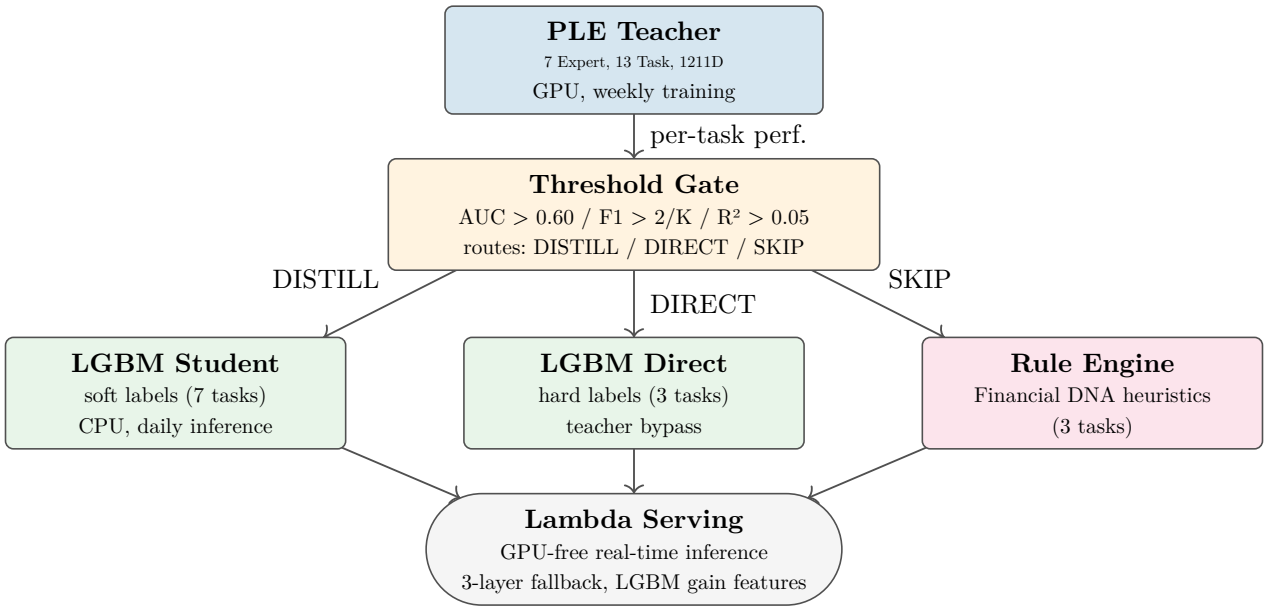

Figure 1: Teacher-student distillation architecture with threshold-gated three-way task routing.

attributions still require translation into business language.

LLM-based generation is an emerging approach that uses large language models to produce natural-language explanations from model outputs. Recent work explores LLM-augmented recommendation explanation, but faces challenges of hallucination, regulatory compliance, and grounding in actual model reasoning.

Gap: No system combines structural model explainability (gate weights) with multi-agent LLM-based generation under regulatory safety constraints, where each agent has a specialized role (selection, generation, validation) and all outputs are audit-logged.

2.3 Responsible AI in Finance

Korean FSS published AI guidelines (2021) [6] and model risk management directives requiring explainability, fairness monitoring, and audit trails. The EU AI Act [3] classifies financial credit/recommendation as high-risk AI, mandating transparency (Art. 13), human oversight (Art. 14), and accuracy (Art. 15). The EBA [2] calls for "interpretable" models in internal risk assessments. Korea's AI Basic Act [13] (passed December 2024, promulgated January 2025, effective January 22, 2026) adds domestic high-impact AI classification.

Pearl [14] argues that true explanation requires causal understanding, not mere statistical association — a position increasingly echoed by financial regulators who demand explanations reflecting the actual decision mechanism [16].

Gap: No recommendation system provides an explicit, verifiable mapping from regulatory requirements to system architecture components, with explanations grounded in causal reasoning rather than post-hoc correlation.

2.4 LLM Safety and Grounding

Deploying LLMs for customer-facing financial text introduces specific risks: hallucination (stating non-existent product features), inappropriate advice (recommending unsuitable products for the customer's risk profile), and regulatory violation (breaching the Financial Consumer Protection Act or the Suitability Principle).

Retrieval-Augmented Generation (RAG) mitigates hallucination by grounding generation in retrieved factual context. Our approach extends this: rather than retrieving from a general knowledge base, we ground generation in *model-internal feature attributions* that have been reverse-mapped to business descriptions. This ensures the generated text reflects what the model actually computed, not what an LLM independently "knows."

Content filtering and safety gates provide a final defense layer. Our 3-agent architecture separates generation from validation, enabling independent improvement of each component.

3 Knowledge Distillation

3.1 Teacher-Student Architecture

The teacher model (PLE with 7 heterogeneous experts, 13 tasks, 1211 features; see companion paper for architecture details) produces soft probability outputs that serve

as training targets for per-task LGBM [10] students. The teacher's value as a distillation source stems from its *structural guarantee* against *expert collapse*: because the seven experts are architecturally distinct (DeepFM, MLP, Mamba, HGCN, PersLay, etc.), they cannot converge to the same function, ensuring the soft labels encode genuinely multi-faceted customer understanding. Note on HGCN: the graph expert receives 27-dimensional `merchant_hierarchy` features (MCC L1 $\rightarrow$ L2 $\rightarrow$ code Poincaré embeddings), not product co-holding features. This distinction matters for distillation: LGBM gain importance from the HGCN-trained student ranks highly for MCC-dependent tasks (e.g., top_mcc_shift), and the top-gain features produce explanation-grounded statements such as "customer shows sustained preference for merchant category X" rather than generic co-holding signals. Crucially, not every task benefits equally from the teacher's soft labels. The adaptive threshold gate (Section 3.1.1) assesses teacher quality per task and routes accordingly: tasks where the teacher exceeds 2x random baseline are distilled, while others fall back to direct hard-label training or the rule engine, ensuring the soft-label signal is only applied where it genuinely adds value.

The key design decision is *per-task distillation*: rather than a single student model for all 13 tasks, we train 13 independent LGBM models, each learning one task's soft labels. This enables: (1) per-task feature selection (different tasks benefit from different features), (2) independent retraining (if one task drifts, only its student is re-distilled), (3) interpretable feature importance per task (LGBM's built-in feature importance aligns with the business reverse-mapping for explanation generation).

Five tasks were excluded from the task set during development. Four (income tier, tenure stage, spend level, engagement score) represent deterministic feature transformations — for instance, income tier is simply a quantile bucket of the raw income feature, which is already a model input. A student model can perfectly reconstruct such labels from its input features, making the distillation trivially solvable and uninformative. The fifth, has_nba (binary "will acquire any product?"), was folded into nba_primary class 0 — predicting *which* product to recommend subsumes predicting *whether* to recommend. The remaining 13 tasks represent genuine prediction objectives where the label cannot be deterministically derived from input features.

Lifecycle separation:

- **Teacher**: retrained weekly/monthly on SageMaker (GPU required, comprehensive). The teacher captures complex non-linear expert interactions through heterogeneous expert gating that LGBM cannot directly learn.
- **Students**: re-distilled daily with fresh soft labels (CPU only, fast). Daily re-distillation tracks data drift without the cost of GPU training.
- **Champion-Challenger**: automatic comparison of new student vs. current production model. If the new student's AUC drops below threshold, the update is blocked and an alert is raised.

This architecture resolves a fundamental tension in financial AI: the *model that learns best* (deep PLE with GPU) is not the *model that serves best* (lightweight LGBM on CPU Lambda). Knowledge distillation [8] bridges this gap, and the FD-TVS (Friedman Decomposition–Transitory Variance Scoring) system [7] further weights the student's predictions by income stability type. FD-TVS applies Friedman's Permanent Income Hypothesis to decompose each customer's observed income into permanent (stable, long-term) and transitory (bonus, irregular) components via HP or Kalman filtering. Customers whose income is predominantly transitory receive lower confidence weights for long-horizon product recommendations, preventing the system from recommending products that assume stable cash flow to customers with volatile income patterns. Beyond income-stability weighting, FD-TVS applies *dynamic task-level weights* during scoring. Base task weights are read from config; these are multiplied by segment-based dynamic multipliers (clipped to 1.0–1.5) and behavior-based rules (e.g., a spike in a specific feature boosts the weight of the correlated task). Both on-premises and AWS deployments share this task-level dynamic weighting design, with the AWS version additionally supporting cloud-native config management via `scoring.segment_task_weights` and `scoring.dynamic_weight_rules`.

Temperature $T = 1$ for tree-based students. The original distillation formulation [8] raises the softmax temperature T to soften the teacher's output distribution, amplifying the relative probabilities of non-target classes ("dark knowledge") so that the student neural network receives gradient signal across all classes. However, this rationale does not transfer to LGBM students: (1) LGBM learns via split-based optimization, not backpropagation, so vanishing gradients on tail classes are not an issue; (2) raising T compresses the probability range — for a binary task with 2.98% positive rate, $T = 5$ maps teacher output from $[0.03, 0.97]$ to $[0.48, 0.52]$, effectively destroying the class-discriminative signal; (3) the Soft GBM study [5] shows that hard CART trees already struggle to learn multi-dimensional soft label vectors, and flattening these further via high T only exacerbates the problem. We therefore set $T = 1$, using the teacher's

Layer	Mechanism	Trigger	Latency
1. Distillation	PLE teacher → LGBM soft labels	Default path	< 10ms
2. Direct Training	LGBM on hard labels (teacher bypass)	Teacher threshold $< 2\times$ random	< 10ms
3. Rule-based	Financial DNA group heuristics	LGBM training failure / drift	< 1ms

Table 1: Three-layer serving fallback architecture. Each layer activates when the preceding layer is unavailable or degraded.

calibrated probabilities as-is and relying on the custom LGBM objective function to inject the soft-label gradient at each split. This design choice is specific to tree-based student models; should a neural student architecture be adopted in the future, temperature scaling can be re-introduced at that point.

3.1.1 Adaptive Distillation Strategy

Not all tasks benefit equally from distillation. *Note: The companion paper (Paper 1) describes the PLE teacher architecture and ablation study; the three-way DISTILL/ DIRECT/SKIP routing logic introduced here is unique to this paper and is not described in Paper 1.* When the teacher's performance on a task falls below approximately twice the random baseline (binary: AUC $<$ 0.60; multiclass: F1-macro $< 2/K$ for K classes; regression: $R^2 < 0.05$), soft labels carry insufficient inter-class information to guide student learning. Note: R^2 is used exclusively as a *gate threshold* to detect near-zero teacher signal on regression tasks; the companion paper reports MAE as the primary regression metric for distillation quality gaps, and this paper follows the same convention (see Table 2, regression rows: MAE gap is the student evaluation metric). In such cases, the pipeline automatically falls back to training the LGBM student directly on hard labels.

This adaptive routing is a Model Risk Management safeguard: rather than propagating low-quality teacher knowledge into production, the system self-diagnoses and activates a safer training path. The fallback preserves service continuity — every task has a serving student regardless of distillation viability — while the monitoring dashboard flags tasks requiring teacher improvement.

In our benchmark experiments, 7 binary tasks exceeded the AUC threshold (0.63–0.72) and were successfully distilled (AUC gap 2–3 percentage points). Two multiclass tasks (nba_primary, segment_prediction) fell below the F1 threshold and were routed to direct hard-label training; the third (next_mcc, 50-class) fell below floor and was routed to SKIP (Layer 3 rule engine). One regression task (mcc_diversity_trend) was borderline (R^2 0.031) and used direct training; the remaining two (product_stability, cross_sell_count) fell below floor ($R^2 < 0.01$) and were routed to SKIP (Layer 3).

3.1.2 Three-Layer Fallback Architecture

The adaptive routing strategy determines whether each task is served by Layer 1 (distillation), Layer 2 (direct hard-label training), or Layer 3 (rule engine) in the broader fallback architecture. If *both* teacher distillation and direct LGBM training degrade beyond acceptable thresholds, the system activates a rule-based fallback (Layer 3) grounded in established financial marketing theory. Each layer is organized by the Financial DNA task groups:

At serving time, Layers 1 and 2 both produce LGBM predictions via the same Lambda inference path; the distinction is in training methodology (soft-label distillation vs. hard-label direct), not in serving architecture.

Layer 3 rules are aligned with the Financial DNA task group taxonomy and grounded in domain-validated heuristics:

- **Engagement group** (churn, behavioral shift): RFM scoring [4] — recency, frequency, monetary value thresholds from operational history.
- **Lifecycle group** (segment, product recommendation): Product adjacency matrix — the empirically observed cross-sell path (deposits → savings → investment → insurance) follows customer lifecycle stages.
- **Value group** (product stability, cross-sell count): CLV-based tiering — customer lifetime value brackets mapped to product complexity tiers, consistent with financial suitability requirements.
- **Consumption group** (MCC patterns, spending diversity): Transaction category frequency analysis informed by Friedman's Permanent Income Hypothesis [7] — distinguishing transitory from permanent consumption shifts.

Critically, all Layer 3 rules respect the regulatory suitability principle (Financial Consumer Protection Act, Article 17): a customer's risk tolerance grade must equal or exceed the product's risk grade. This constraint is *always* enforced regardless of which layer generates the recommendation, providing a regulatory floor beneath the entire architecture.

The three-layer design ensures that service never halts due to model failure — a key operational requirement in

financial production systems and a core principle of SR 11-7 model risk management [1].

3.1.3 Lambda Serving Integration

All three layers are integrated into a unified Lambda serving path:

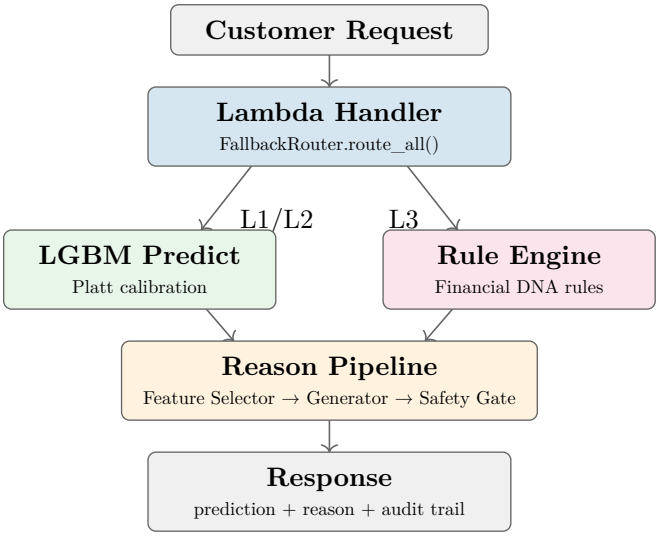

Figure 2: Lambda serving integration. FallbackRouter auto-routes to Layer 1/2 (LGBM) or Layer 3 (rules).

Key integration properties:

- `FallbackRouter` auto-routes each task to its appropriate layer based on availability and metric thresholds; the caller is unaware of which layer served the prediction.
- Calibrated probabilities (Platt scaling) are applied on Layer 1/2 outputs for probability-critical binary classification tasks where raw LGBM probabilities are systematically biased. The calibration-enabled task set is declared per dataset under the distillation configuration (e.g. `distillation.calibration.tasks`) so that probability-consuming downstream logic (thresholding, expected-value scoring, risk budgeting) is correctly calibrated regardless of benchmark. Ranking-oriented binary tasks that only require a consistent score ordering, and regression tasks routed to Layer 3 (rule engine), do not receive Platt scaling.
- For Layer 3, `contributing_features` from rule firings are injected directly into the reason pipeline. This enables interpretable reasons even when no model is available, using the same 3-agent interface as Layer 1/2.
- All three layers produce an identical response schema — prediction, probability, contributing features, reason text, audit token — ensuring the API contract is transparent to callers regardless of which layer served the request.

3.2 Feature Selection

Feature selection for per-task LGBM students is performed using LGBM's native gain importance, computed on the trained student model. This design aligns feature selection with the serving model itself: since LGBM is the production model (not the PLE teacher), gain importance from the LGBM student is the correct signal for identifying which features actually drive student predictions.

Features are ranked by cumulative gain importance and the top-k features capturing approximately 95% of cumulative gain are retained per task. The resulting feature set is typically 40–80 features per task (down from 1211).

This approach has three advantages over teacher IG attribution: (1) **Serving model alignment**: LGBM gain reflects what the deployed model computes, not what the teacher model computed during training — these may differ substantially given the architectural gap between deep PLE and gradient-boosted trees. (2) **Operational stability**: LGBM gain computation is a fast post-hoc step on the trained student; teacher IG requires backpropagation through the full PLE graph at inference time, causing out-of-memory failures at production scale (941K customers, 1211 features). (3) **Interpretability alignment**: SHAP/gain explanations produced from the LGBM student are directly grounded in the model that generated the recommendation, satisfying the EU AI Act Art. 13 requirement that explanations reflect the actual decision mechanism.

For explanation generation (Section 4), LGBM SHAP values at serving time identify the top contributing features for each customer's recommendation. These are then reverse-mapped to business-interpretable language via the interpretation registry.

3.3 Distillation Results

Benchmark v12 produces meaningful label distributions across all 13 tasks. Notably, `nba_primary` reaches 60% no-NBA (down from 91% in earlier benchmarks) and `top_mcc_shift` reaches 50% positive rate (down from 92%), removing the near-degenerate class imbalance that previously made teacher-student fidelity numbers artificially easy to achieve. These distributions make the distillation gaps below genuinely informative.

Teacher-student fidelity metrics are reported per task type, chosen to match each task's production semantic. Binary classification tasks use AUC gap (threshold-independent and imbalance-robust). Multiclass tasks use F1-macro as the *routing metric* (threshold gate decision); NDCG@K and top-K accuracy [9] are reported separately for recommendation-type multiclass tasks

Task	Teacher	Student	Gap	Agree.
Binary (distilled via soft labels)				
churn_signal (AUC)	0.687	0.665	0.022	88.9%
will_acquire_accounts (AUC)	0.721	0.697	0.024	92.5%
will_acquire_payments (AUC)	0.693	0.661	0.032	90.8%
will_acquire_deposits (AUC)	0.671	0.653	0.018	79.8%
will_acquire_investments (AUC)	0.675	0.652	0.023	79.7%
will_acquire_lending (AUC)	0.666	0.640	0.026	81.2%
top_mcc_shift (AUC)	0.630	0.594	0.036	99.9%
Multiclass (threshold-routed)				
nba_primary (F1-macro, 7-cls)	0.187	0.373	0.186	F1 $< 2/7 \rightarrow$ DIRECT
segment_prediction (F1-macro, 4-cls)	0.403	0.376	0.028	F1 $< 2/4 \rightarrow$ DIRECT
next_mcc (F1-macro, 50-cls)	0.012	—	—	F1 $<$ floor $\rightarrow$ SKIP (L3)
Regression (threshold-routed)				
product_stability (R^2)	$<$ floor	—	—	$R^2 <$ floor $\rightarrow$ SKIP (L3)
mcc_diversity_trend	R^2=0.031^{R}	MAE 0.025^{E}	n/a	$R^2 < 0.05 \rightarrow$ DIRECT
cross_sell_count (R^2)	0.008	—	—	$R^2 <$ floor $\rightarrow$ SKIP (L3)

Table 2: Distillation results per task. Binary tasks use AUC gap (evaluation metric). Multiclass tasks use F1-macro as the *routing* metric (threshold: $2/K$); NDCG@K is reported separately in per-task analysis. Regression tasks use R^2 as the *routing* metric (superscript R) and MAE as the *evaluation* metric (superscript E); gap is marked n/a when routing and evaluation metrics differ. DIRECT-routed tasks show hard-label LGBM results; SKIP-routed tasks are served by the Layer 3 rule engine. **Note: Teacher AUC values (0.63–0.72) reflect the v13 phase0 teacher; the v14 SageMaker re-run with HMM mode-split + GMM K=14 + probability-column normaliser fix lifts the teacher's per-task AUC into the 0.82+ range (see Paper 1 joint-ablation and epoch-budget tables). The fidelity gap and student-teacher agreement percentages above are expected to remain in the same magnitude because Platt scaling and gain-importance feature selection are scale-invariant; LGBM distillation on the v14 teacher is deferred to a follow-up release.**

(nba_primary, next_mcc) in per-task analysis. Regression tasks use R^2 as the *routing metric* and MAE gap as the *evaluation metric*. This avoids conflating metrics with incompatible semantics across task types.

The interpretation registry serves as the **data contract** between distillation and reason generation. Distillation (Section 3) decides *which* features to preserve in the student model — selecting the top-gain features that capture 95% of cumulative LGBM importance per task. Reason generation (Section 4) decides *how* to explain those features to humans — mapping each preserved feature to business-interpretable language via LGBM SHAP attributions. This separation means the two stages can evolve independently: improving feature selection does not require rewriting explanation templates, and vice versa.

4 Recommendation Reason Generation

4.1 Feature Business Reverse-Mapping

Every feature in the system is registered in the interpretation registry with structured business metadata:

This registry serves dual purposes: (1) grounding material for the Reason Generator agent, and (2) audit trail showing which features influenced each recommendation.

The interpretation registry interprets features into Korean via a 5-level cascade (highest priority first):

- **Level SHAP** (highest) — SHAP sign direction + task context
- **Level 3** — feature×task manual overrides
- **Level 2** — group×task
- **Level 1** — group×task_group auto-generated

Feature	Business Mapping	Explanation Template
hmm_lifecycle _prob_growing	Growth stage probability	"Your asset portfolio is in a growth phase"
mamba_temporal _d3	3-month spending trend	"Your spending has been increasing recently"
hgcn_hierarchy _d5	Product category position	"You belong to a segment that actively uses card benefits"
synth_stability	Transaction stability	"You maintain a stable transaction pattern"
gmm_cluster _prob_3	Segment probability	"You share patterns with customers of similar lifestyles"

Table 3: Feature reverse-mapping examples.

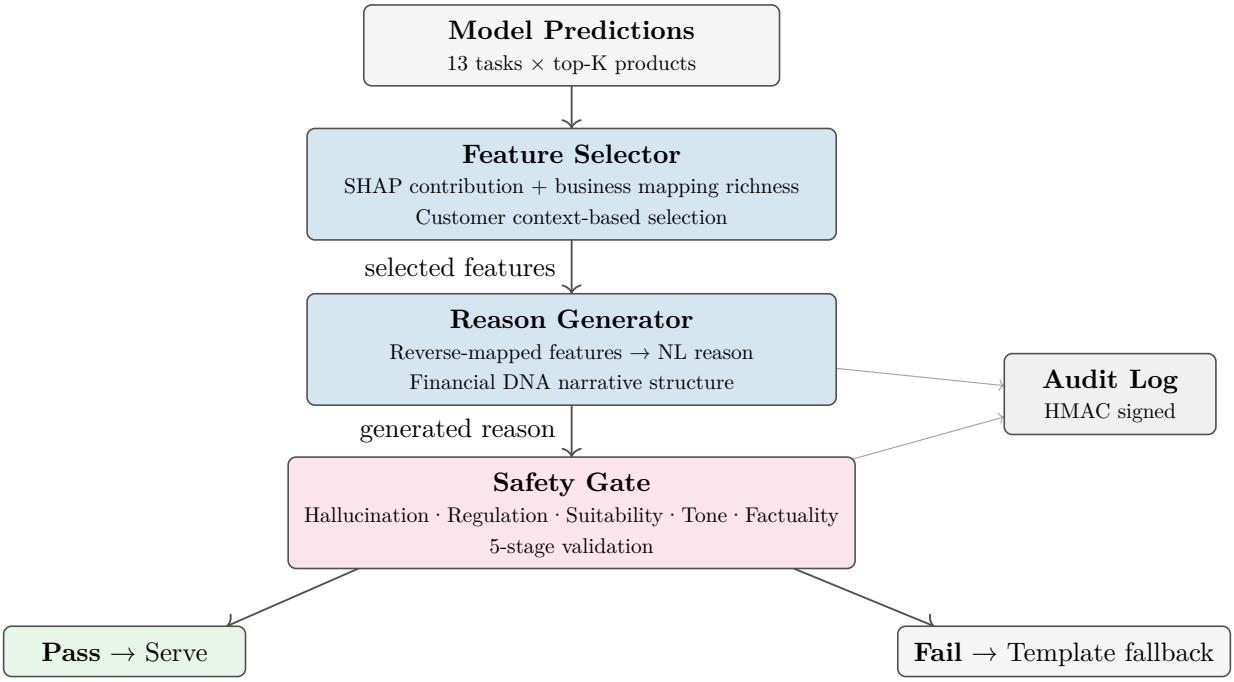

Figure 3: 3-agent recommendation reason generation pipeline.
Feature Selector → Reason Generator → Safety Gate.

- **Level RM** (lowest) — reverse-mapping layer glossary templates

Each level is tried in order; the first match produces the interpretation.

Only features unresolved by this cascade fall to raw fallback. The reverse-mapping layer is integrated as Level RM, so glossary value-substitution templates (e.g., "average {value} monthly transactions") operate as part of the cascade. All fallback text outputs are generated in Korean; English translations are shown throughout this paper. Korean originals are available in the public repository.

4.2 3-Agent Pipeline Architecture

The three agents map to their implementation classes as follows: (1) **Feature Selector** (implemented as `FactExtractor`), (2) **Reason Generator** (implemented as `InterpretationRegistry` + `TemplateEngine`), and (3) **Safety Gate** (implemented as `SelfChecker`). In the remainder of this section, marketing names are used in prose and implementation names appear only in technical/code contexts.

4.2.1 Agent 1: Feature Selector

Implemented as `FactExtractor`, this agent selects features for explanation based on:

- LGBM SHAP values at serving time (model-driven relevance, aligned with the serving model)

Check Category	Criteria
Hallucination	No claims about facts not in feature data
Regulatory	No violation of KFCPA §19 (duty of explanation — no false or exaggerated explanation)
Appropriateness	No unsuitable product recommendations for customer profile
Tone	Professional financial advisory language
Factual	Numerical claims match actual feature values

Table 4: Safety Gate validation criteria.

- Business reverse-mapping richness (explanation-driven relevance)
- Customer context (personalization: different features matter for different customer profiles)

The Feature Selector uses the Financial DNA axis from the companion paper's two-axis framework to select features from the relevant customer dimension (e.g., lifecycle features for a retention recommendation, spending-pattern features for a benefit recommendation), ensuring explanations address the dimension most pertinent to the recommended action. For Layer 3 (rule-based) predictions, `contributing_features` from rule firings are passed directly to this agent. This enables the same agent interface regardless of which serving layer produced the prediction.

4.2.2 Agent 2: Reason Generator

Implemented as `InterpretationRegistry` + `TemplateEngine`, this agent receives selected features with their business reverse-mappings and generates a natural-language recommendation reason. The `InterpretationRegistry` reverse-maps each feature name to Korean business descriptions via the 5-level cascade (SHAP direction → L3 → L2 → L1 → reverse-mapping layer); the `TemplateEngine` then assembles these into a coherent recommendation reason. The Reason Generator structures its narrative around Financial DNA groups — opening with the customer's current state (lifecycle), grounding the recommendation in observed behavior (engagement/consumption), and framing the value proposition in terms the customer recognizes (value dimension) — producing what the companion paper calls *multi-faceted customer understanding* rendered as persuasive text. Grounding constraints:

- Must reference only features actually present in the selection.
- Must use business terminology appropriate for the target audience (customer vs. relationship manager).
- Must follow financial domain tone and compliance guidelines.

L1 (template, 0.1ms):
"Benefits tailored to your changing spending patterns. We recommend customized spending benefits for your diverse category usage."

L2a (Bedrock Claude Sonnet rewrite, 2.4s):
"We recommend a customized spending benefit product so you can enjoy practical benefits across multiple categories."

SelfChecker verdict: pass

Figure 4: Actual Lambda production output for task top_mcc_shift (customer 10). L1 template in 0.1ms; L2a Bedrock rewrite cached at 6ms. English translations are shown above; Korean originals are in the public repository.

4.2.3 Agent 3: Safety Gate

Implemented as `SelfChecker`, this agent validates the generated reason against:

On failure: automatic fallback to template-based safe reason. All gate decisions are logged for audit trail. Upstream of the 3-agent pipeline, the constraint engine applies eligibility and suitability filters — verifying customer context, usage patterns, and product-specific constraints — so that no recommendation reaches the customer without passing a suitability check, as required by the Korean Financial Consumer Protection Act (금소법, KFCPA) Article 17.

4.2.4 Serving Model Selection

Customer-facing recommendation reasons require natural, professional Korean text. The optimal model differs by deployment environment:

- **On-premises (air-gapped)**: Exaone 3.5 7.8B (LG AI Research, Apache 2.0) — Korean-specialized training produces more natural financial honorific tone than same-class models (Llama, Qwen). Runs on RTX 4070 12GB.
- **Cloud (AWS)**: L2a rewriting uses Bedrock Claude Sonnet — natural Korean generation with Bedrock-native availability (no Marketplace onboarding required). L2b self-critique also uses Claude Sonnet

(generator ≤ critic model principle). The self-check layer's factuality scoring uses Claude Haiku. Ops/Audit agents use Claude Sonnet. All invocations use cross-region inference profiles (`us.anthropic.*`).

Bedrock ensures that input/output data is never transmitted to model providers (Anthropic) and is never used for model training. VPC PrivateLink enables invocation without traversing the public internet. Cross-region inference profiles (e.g., `us.anthropic.*`) route requests to the geographically optimal endpoint while Bedrock guarantees that *customer data in the API payload* is processed within the caller's contracted data boundary and is not persisted by the provider. Financial customer identifiers (account numbers, resident IDs) are stripped by `PIIEncryptor` before any data enters the LLM prompt, so the prompt contains only anonymized behavioral features. This layered approach — PII stripping at the application boundary plus Bedrock's provider-side data isolation — is designed to support the data governance requirements of Korean FSS AI guidelines and the Personal Information Protection Act.

The LLM backend is config-driven, allowing the deployment environment (Bedrock, local open-source, or mock) to be switched without code changes.

The reason generation pipeline uses its own tier nomenclature (L1/L2a/L2b), distinct from the serving fallback layers (Layer 1/2/3 in Table 1) which refer to model training methodology. The three reason generation tiers map to Bedrock invocation as follows:

- **L1 (synchronous)**: Template-based, 1ms latency, always available. No Bedrock call; the TemplateEngine generates deterministic Korean from InterpretationRegistry reverse-mappings. This is the guaranteed fallback for all customers.
- **L2a (on-demand)**: When a customer clicks for detail, a synchronous Bedrock Claude Sonnet call rewrites the L1 template into richer, more natural Korean (first call 2.4s); the result is cached in DynamoDB so subsequent requests return instantly (6ms cache hit). Processing priority is determined by context richness (data availability), not customer tier, satisfying the equal-explanation obligation (KFCPA §19).
- **L2b (async)**: Bedrock validates L2a output for PII leakage, hallucination, and regulatory compliance before promotion. Human review is applied to a 5% sampling for quality assurance.

The Bedrock infrastructure is shared between reason generation and operational agents (Section 5); time-slot separation resolves quota contention.

4.2.5 Fact Compression Layer (Mem0 Adoption)

While the interpretation registry provides feature-level interpretation, the fact extraction layer adds **customer-level narrative facts**. A rule-based engine extracts Korean facts from feature values — deterministically and without any LLM calls. Example facts include:

- "Deposit-focused portfolio"
- "Recent 3-month card usage growth"
- "Risk-averse tendency"

These facts are extracted at serving time via config-driven rules (15 categories) and injected into the L2a prompt as a "Customer Facts" section. The L2a model (Claude Sonnet on AWS; Exaone 3.5 on-premises) then generates reasons **with customer understanding**, not just raw feature values.

Rules are defined in a YAML configuration file (15 categories covering portfolio composition, interests, risk tolerance, lifecycle, etc.) and new facts can be added with config-only changes.

4.3 Caching Strategy and Asynchronous L2a Architecture

Recommendation reasons are served via a 3-layer asynchronous architecture:

1. **L1 (Template)**: returned immediately on customer request. No LLM call. The template engine generates deterministic Korean reasons based on LGBM SHAP top-K feature business reverse-mappings. Features pass through the interpretation registry's 5-level cascade (SHAP direction → L3 → L2 → L1 → reverse-mapping layer) to produce enriched 3-tuples `(feature_name, shap_value, Korean_interpretation)`.

2. **L2a (on-demand with caching)**: When a customer clicks for product detail, a synchronous Bedrock Claude Sonnet call (AWS) or Exaone 3.5 (on-premises) refines the L1 reason into richer natural Korean. The first call takes 2.4s (Bedrock Sonnet); the result is cached in DynamoDB so the next request for the same customer×product returns at 6ms. This on-demand pattern replaces an earlier SQS-based asynchronous design — it provides an immediate response on click rather than requiring the customer to wait for a background queue, and caching makes repeat access instant. **All customers receive the L1 template equally**, and L2a invocations are triggered by **customer-initiated detail requests** rather than customer tier — complying with Korean Financial Consumer Protection Act Art.19 (equal explanation obligation) and Personal Information Protection Act

Art.37-2(2) (right to explanation). Context richness classification: rich (abundant features + history) → moderate (partial features) → sparse (cold-start; excluded from L2a, L1 template only). This reflects that **data availability** determines LLM output quality, not a service-quality differential by customer segment.[1]

3. **L2b (Quality Validation)**: applies a 5-stage safety gate to L2a output — (1) prompt sanitizer, (2) PII detection (Korean resident registration number, card numbers, etc.), (3) self-check layer (compliance + injection + factuality), (4) grounding verification (number cross-check), (5) 5% human review sampling. Pass promotes to L2b; failure falls back to L1.

Caching uses a dual backend (in-memory + DynamoDB) with composite key `customer_id + product_id + task_name` and TTL-based auto-expiry. Of 941K customers, L2a targets (5% sample, 47K items) are processed by 5 parallel Claude Sonnet workers in 8 minutes at $0.21 cost (47K × 500 input + 200 output tokens at Claude Sonnet pricing).

4.3.1 Serving Security

Financial customer data requires PII protection both inbound (raw feature inputs) and outbound (generated recommendation text). Two modules enforce this at the serving boundary:

- **`PIIEncryptor` (inbound)**: scans feature input vectors for PII patterns (resident registration numbers, card numbers, account numbers) before inference. Detected PII fields are hashed in-place using HMAC-SHA256 with a per-customer salt, so model inference operates on anonymized values. Controlled via `SECURITY_FEATURE_SCAN=true` environment variable.
- **`PromptSanitizer` (outbound)**: scrubs PII from generated recommendation reason text before it is returned to the customer interface. Additionally, `PromptSanitizer` classifies prompt sensitivity into three tiers — HIGH (contains financial identifiers or account-level data), MEDIUM (contains behavioral patterns), LOW (generic recommendation context) — and routes accordingly: HIGH prompts are sent exclusively to Amazon Bedrock (data stays within the AWS region), MEDIUM and LOW may be routed to alternative providers (e.g., Gemini) only after PII has been stripped and the prompt contains no customer-identifiable information. This tiered routing applies to *prompt context*, not to raw customer data, which never leaves the Bedrock boundary. Controlled via `SECURITY_PII_SCRUB=true` environment variable.

This two-layer boundary ensures that neither raw PII nor LLM-generated outputs that contain PII can escape the serving perimeter, designed to align with the data minimization principle of GDPR and Korean PIPA.

5 Operational Agent Pipeline

The preceding section described three serving agents that generate recommendation reasons on the customer-facing path. This section introduces two additional agents that operate on a separate, batch-only path: interpreting monitoring outputs and compliance reports so that a small operations team can maintain regulation-compliant MLOps without dedicated MLOps staff.

5.1 Motivation: Human–AI Role Separation

Not all operational tasks require the same cognitive mode. Formalized, repetitive monitoring — checking whether PSI exceeded a threshold, whether fairness metrics dipped below a limit, whether gate entropy shifted — is well-suited to LLM agents that run on a batch schedule (daily or weekly). Non-routine judgment — diagnosing data contamination, interpreting business context shifts, responding to regulatory interpretation changes — requires human expertise that no current LLM can reliably provide.

The key design insight is role separation: agents tell *what changed*; humans decide *why it changed and what to do*. This division makes regulation-compliant operations feasible for small teams (1–2 people) without dedicated MLOps staff, because the agents eliminate "dashboard fatigue" — instead of checking 10 metrics daily, the human reads one natural-language summary.

5.2 Two Operational Agents

5.2.1 OpsAgent (Operations Agent)

The OpsAgent runs after training completion and drift monitoring DAG executions.

Inputs: evaluation metrics, training logs, CGC gate entropy, PSI drift reports.

Output: A "Model Health Report" in natural language.

[1] An earlier prototype set L2a priority based on customer segment (VIP), but was redesigned around context richness classification to comply with the Korean Financial Consumer Protection Act Art.19 equal-explanation obligation (on-prem v3.0.0, 2026). This design transition is continuously monitored under AV1 (fairness) of the audit agent.

OpsAgent CP4 finding (measured):
"Distillation fidelity gap 0.1858 > threshold 0.05. 8/10 tasks exceeded calibration gap. Possible teacher model change or feature distribution shift. Recommend comparing teacher-student prediction distributions for affected tasks."
3-agent consensus: 3/3 FAIL (unanimous)

Figure 5: OpsAgent Model Health Report — actual CP4 distillation finding with 3-agent consensus verdict. Report generated in Korean; English translation shown above.

Triggers: drift monitoring DAG completion, training job completion. Stage completion events are emitted by `ChangeDetector` at the end of every pipeline stage (training, eval, distillation, serving, reason generation), providing the event stream from which OpsAgent selects its input window. A `{"action": "heartbeat"}` endpoint on the Lambda handler returns system health status without running inference, allowing the OpsAgent to confirm serving availability before submitting a health report. Model version changes are tracked at Lambda cold start: when the champion model version changes, an event is emitted so the OpsAgent can flag the transition in its next report.

The OpsAgent does not make promotion decisions — it surfaces the information a human needs to make one. When all metrics are within normal bounds, the report is brief ("All 13 tasks within tolerance; no action required"). When anomalies are detected, the report identifies *which* tasks and *which* experts are affected, providing actionable context rather than raw numbers.

5.2.2 AuditAgent (Audit/Compliance Agent)

The AuditAgent runs after fairness monitoring and governance DAG executions.

Inputs: fairness monitor reports (DI/SPD/EOD), audit trail integrity checks, opt-out statistics, governance checklist status.

Output: A "Regulatory Compliance Report" in natural language.

AuditAgent finding (measured):
"Recommendation reason grounding score 0.33 (threshold 0.50): only 1 of top-3 tasks had keyword match. Bias DI = 1.0 (4 protected groups treated equally). KFCPA violations: 0 (5 rules verified)."
3-agent consensus: grounding FAIL (1W+2F), fairness WARN (2P+1W), KFCPA PASS (3/3 unanimous)

Figure 6: AuditAgent Regulatory Compliance Report — actual measured results with consensus verdicts. Report generated in Korean; English translation shown above.

Triggers: fairness monitoring DAG completion, quarterly governance cycle.

The AuditAgent converts quantitative fairness metrics into regulatory language, explicitly referencing the applicable regulation (FSS guideline number, EU AI Act article) so that the human reviewer can act without cross-referencing documentation. `AuditLogger` records training and distillation completion events as immutable audit entries, ensuring that the AuditAgent's input is a tamper-evident, time-ordered record of pipeline activity.

5.3 Design Principles

The critical constraint is that operational agents have *no shared state* with the serving path. The serving pipeline (Feature Selector → Reason Generator → Safety Gate) produces customer-facing outputs; the operational pipeline (DAG completion → OpsAgent/AuditAgent → report storage → human review) produces internal operations artifacts. This separation ensures that an operational agent failure can never degrade customer-facing service.

5.4 Model Selection for Operational Agents

Unlike serving agents, which require Korean-language fluency for customer-facing text, operational agents process structured JSON inputs and produce logical assessments — natural language fluency is secondary to reasoning accuracy. Model assignment by deployment environment:

- **On-premises (air-gapped)**: Exaone 3.5 7.8B (Korean reason generation), Qwen 2.5 7B (tool-calling agent reasoning), Exaone 3.5 2.4B (interpretation and consensus), and bge-m3 (embeddings). Sequential loading on RTX 4070 12GB VRAM. An internal four-scenario benchmark selected the 7B model over the previously used Qwen 2.5 14B Q4 for the tool-use role: the smaller model completed all three tool-call scenarios that the quantised 14B failed — a larger model is not automatically better at tool calling.

Principle	Rationale
Batch-only, never real-time	No serving path dependency; agents run asynchronously after DAG completion
Per-task optimal model assignment	• Reason generation: Claude Sonnet (AWS) / Exaone 3.5 (on-premises) • Agent dialog/consensus: Claude Sonnet (contextual reasoning, Ops/Audit agents included) • Factuality judgment: Claude Haiku (low cost) • Embeddings: Titan V2 • On-prem: Exaone 3.5 7.8B (reasons) + Qwen 2.5 7B (tool use) + Exaone 3.5 2.4B (consensus)
Reports deposited to shared folder	Alerts via Slack/email only on anomalies; human reviews at their own pace
Agent outputs are audit artifacts	Immutable, HMAC-signed; the report itself is evidence of monitoring
Cost: $0.03/day (3× consensus)	1–2 small-model calls with structured input per execution cycle

Table 5: Operational agent design principles.

- **Cloud (AWS)**: per-task optimal models are assigned.
  - Claude Sonnet (cross-region inference profile) — Korean L2a reason rewrite + self-critique (Exaone 3.5 for on-premises)
  - Claude Sonnet — ops/audit agent dialog, 3-agent consensus
  - Claude Haiku (cross-region inference profile) — self-check layer factuality judgment
  - Claude Opus — quarterly deep audit
  - Titan Embeddings V2 — vectorization

  The Bedrock infrastructure is shared between reason generation and agents; quota competition is resolved via time-slot separation.

In both deployments, operational agents execute only 1–2 calls per DAG cycle, keeping cost and latency negligible regardless of model choice.

5.5 Practical Value

The operational agents address three concrete pain points in financial AI operations:

1. **Dashboard fatigue elimination**: Instead of monitoring 10+ metrics dashboards daily, the operations team reads one natural-language summary — "check when you come in to work."
2. **Automatic audit evidence accumulation**: When regulators ask "How did you respond to drift event X?", the institution produces the OpsAgent's report from that date plus the human's subsequent action record, forming a complete incident response trail.
3. **Small-team MLOps enablement**: The architecture makes regulation-compliant operations feasible without a large dedicated MLOps team. The agents handle the formalized interpretation; humans contribute the judgment that regulations actually require.

5.6 Pipeline Inspection Checklist

For systematic inspection, the pipeline is divided into six parts with 48 checklist items: P1 (Ingestion), P2 (Feature Engineering), P3 (Training/Distillation), P4 (Serving/Recommendation), P5 (Reason Generation), P6 (Monitoring/Governance). Each item is defined in YAML config with tool name, threshold, and verdict logic; OpsAgent handles 23 items grouped under 7 operational checkpoints (CP1–CP7, referenced in Table 7), and AuditAgent handles 25 items grouped under 5 audit viewpoints.

5.7 Tool Calling Architecture

39 tools (30 Query + 9 Action) are defined via JSON Schema, wrapping existing monitoring components (drift detector, fairness monitor, self-check layer, etc.) as callable tools for the agents. Query tools can be called freely, while Action tools (incident creation, audit logging) require explicit approval, structurally enforcing the Query/Action boundary.

Three hardening mechanisms sit between the agents and this tool surface. First, *focus-keyed dynamic tool routing*: rather than exposing all 39 tools on every call, the dialog layer selects the subset relevant to the current inspection focus from a routing table (`agent_tool_routing.yaml`, with a built-in fallback map when the file is absent), shrinking the prompt and reducing wrong-tool selection.

Second, *signature-based parameter filtering*: before any tool executes, its arguments are filtered against the wrapped function's actual signature (`tool_registry.py::_filter_params`), so an LLM that hallucinates an extra argument degrades to a dropped

parameter rather than a `TypeError` aborting the inspection mid-run.

Third, *warning triage*: warning-level tool outputs are classified IGNORE / MONITOR / FIX_NOW before any narrative is generated, and a metric on a rising trend is escalated to FIX_NOW even while still below its alert threshold. This extends the SR 11-7 quality-triage principle applied at the distillation gate — monitor outputs, isolate degradation early — to the operational monitoring layer itself.

5.8 Verdict Layering and Agent Self-Verification

Two design rules keep the LLM interpretation layer auditable. First, *verdicts belong to the rule engine*: every PASS/WARN/FAIL verdict is computed by deterministic, reproducible threshold rules with a full audit trail, and the LLM layer (plus the consensus mechanism described below) only interprets, prioritises, and narrates those verdicts. An auditor can replay any verdict without an LLM in the loop.

Second, *self-verification before reporting*: a `verify_grounding` pass re-reads the dialog history and checks each factual claim in the draft conclusion against the actual tool results recorded there, returning a structured `{grounded, issues}` verdict, and a failed check downgrades the conclusion to a flagged draft for human review. The pass is implemented in both environments: enabled by default in the on-premises reference system, and on AWS exposed as a per-call flag on the dialog layer (off by default at the API level); the production report path enables it by default whenever the opt-in LLM follow-up wiring is active.

Log analysis follows the same economy of LLM use: log files are scanned incrementally (only lines appended since the last inspection), ERROR lines escalate to FIX_NOW deterministically with no LLM involvement, and only WARNING lines — where severity is genuinely ambiguous — are passed to the LLM for triage.

5.9 Environment-Adaptive Consensus Mechanism

To structurally mitigate hallucination risk in LLM-based interpretation, the consensus mechanism is *adapted to the deployment environment* because model capability differs fundamentally between the cloud and on-premises settings. A single consensus design cannot serve both: cloud-scale models can afford independent parallel voting, whereas smaller on-premises models require structural safeguards against conformity bias.

5.9.1 AWS: Independent Parallel Voting (Jury Model)

On AWS, three independent Claude Sonnet sessions run in parallel. Each agent is assigned a different perspective: α (conservative), β (statistical significance), γ (business impact). Agents do not see each other's outputs — the verdict is formed post-hoc by aggregation. Latency is 5 seconds per checkpoint and cost is 3× a single Sonnet call.

5.9.2 On-Premises: 2-Round Hybrid (Independent Vote → Sequential Deliberation)

On-premises deployment runs consensus on Exaone 3.5 2.4B, with Qwen 2.5 7B handling tool-use reasoning, on a single RTX 4070 (12GB VRAM). At this parameter scale, a purely sequential deliberation (Delphi-style) exhibits *convergence bias*: later agents anchor to earlier opinions and minority dissent disappears. In operations and audit contexts, a missed signal is far more costly than a false alarm, so we split the process into two rounds:

- **Round 1 (independent vote, 5 agents)**: each agent votes without seeing others' outputs. Minority opinions are *locked* at the end of this round and cannot be deleted in subsequent processing.
- **Round 2 (sequential deliberation, 2 agents)**: two additional agents read the full Round 1 output, strengthen the arguments for each position (majority and minority alike), and produce a final structured verdict. Round 2 improves argument quality without overwriting the minority view preserved from Round 1.

For each checkpoint, Round 1 runs its five votes sequentially under the single-GPU constraint and Round 2 adds two deliberation passes; only items flagged WARN or FAIL by the rule engine enter consensus (typically 5–10 per inspection). With the 2.4B consensus model, per-vote latency is well below the earlier 14B Q4 configuration (which required 45 minutes per inspection cycle), keeping a full cycle comfortably within operational tolerance for a batch-only, post-DAG process.

This hybrid design is chosen over pure Delphi for four reasons: (1) *minority preservation* — secured by Round 1 independence and structurally unmodifiable thereafter; (2) *argument quality* — achieved by Round 2 deliberation, matching the benefit of pure Delphi; (3) *weak-model fit* — Round 1 independence avoids the conformity bias to which small models are especially susceptible; (4) *audit suitability* — every opinion is preserved, so an auditor can always trace "why a minority view was (or was not) escalated."

Tier	AWS (3 agents)	On-prem (5 R1 agents)	Action
Consensus (unanimous)	3/3 agree	5/5 agree	Confirmed verdict
Majority (priority review)	$\geq 2/3$	$\geq 3/5$	Immediate operator review
Minority Report	1/3 dissent	1–2/5 dissent	Secondary review list, preserved separately

Table 6: Verdict classification by environment. The on-prem R1 population is larger (5 vs 3) to compensate for lower per-agent reasoning capacity; the additional R2 deliberation does not alter the locked R1 verdicts.

OpsAgent Checkpoint	Votes	Verdict	Minority
CP2: Phase 0 data quality	3/3 FAIL	FAIL	none (unanimous)
CP4: Distillation fidelity	3/3 FAIL	FAIL	none (unanimous)
CP6: Serving latency 120ms	1 WARN + 2 PASS	WARN	α (conservative)

Table 7: OpsAgent 3-agent consensus results. CP2 and CP4 unanimous FAILs indicate synthetic data limitations (expected in benchmark context). CP6 WARN reflects α agent's conservative threshold for 120ms warm latency.

AuditAgent Item	Votes	Verdict	Minority
Reason grounding (score 0.33)	1 WARN + 2 FAIL	FAIL	α (less strict)
Fairness DI = 1.0	2 PASS + 1 WARN	WARN	α (conservative)
KFCPA compliance	3/3 PASS	PASS	none (unanimous)

Table 8: AuditAgent 3-agent consensus results. Grounding score 0.33 triggers FAIL (1 of 3 sampled tasks). Fairness DI = 1.0 is ideal but α flags small-sample caveat. Financial Consumer Protection Act (금소법, KFCPA) compliance: unanimous PASS, 5 rules, 0 violations.

5.9.3 Common Classification Rules

Both environments produce three verdict tiers (counts differ by agent population):

The core principle across both environments is **minority report preservation**. Novel problem types are often caught first by the dissenting perspective while the majority, anchored to familiar patterns, overlooks them.

Note on independence: in the AWS variant, this pipeline invokes the **same** Sonnet model three times with different system prompts and sampling temperature variation. What this secures is **conditioned diversity**, not weight-level independence. Unanimity therefore indicates only that three conditioned perspectives converged to the same point; a shared training lineage can share the same bias, so we **treat unanimity as a weak signal**. High-risk checks (AV1 fairness, AV2 PII detection, etc.) escalate any minority dissent to human review regardless of the majority verdict. The on-prem variant likewise runs all consensus votes on a single small-model checkpoint, so the same caveat applies; Round 1 independence mitigates but does not eliminate the shared-lineage bias.

5.9.4 Consensus Results from Production Test (AWS variant)

Table 7 and Table 8 report the consensus outcomes from a production test run on the live Lambda serving pipeline (AWS 3-agent variant). On-prem 2-Round results are not reported here because the on-prem deployment is an operational fallback target rather than a benchmark configuration. The verdict rule is: **PASS requires unanimous (3/3)**; any dissent yields WARN; any FAIL vote yields FAIL verdict regardless of majority.

5.10 Diagnostic Case Store

Inspection reports are not disposable artifacts but accumulate as an operational knowledge base. A LanceDB-based diagnostic case knowledge base stores structured metadata (part, item, verdict, severity) and text embeddings (finding + cause + action) for three purposes: (1) **similar case search**: referencing past response history when a new anomaly occurs, (2) **statistical analysis**: aggregating frequent WARN types per part, mean resolution time, (3) **resolution tracking**: quantifying actual effectiveness of responses via (problem, action, post-action verdict) triples.

The case schema includes a `consensus_type` field to track the rate at which minority opinions turned out

to be correct. This data serves directly as "continuous improvement evidence" for regulatory audits.

5.11 Selective Memory Framework Adoption

Core patterns from several 2026 agent memory frameworks (Mem0, Zep/Graphiti, Letta, SuperLocalMemory) were **selectively adopted** as incremental additions to existing infrastructure.

Adoption 1 — Temporal Knowledge Graph (Zep/Graphiti): The temporal fact store uses a `(entity, attribute, value, valid_from, valid_to)` schema for audit evidence. Point-in-time queries like "What was customer A's state at 2026-03-15?" resolve as single filters. All three stores — recommendation cases (LGBM top features + generated reasons, accumulated per served request), diagnostic cases (OpsAgent inspection results), and temporal facts — share the same LanceDB backend, achieving zero new dependencies. LanceDB's combined vector search + column filtering enables queries like "find similar customers as of date X" in a single call. Embedding model choice matters disproportionately for Korean text: in our evaluation, all-MiniLM embeddings collapsed Korean sentence pairs to cosine similarity ≈ 1.0 regardless of content (no discriminative power), whereas bge-m3 separated similar pairs (0.76) from unrelated pairs (0.41). The on-premises deployment therefore standardises on bge-m3, with Titan Embeddings V2 serving the same role on AWS.

Adoption 2 — Mathematical Decay (SuperLocalMemory): The diagnostic case knowledge base's similar case search now applies $\exp\left(-\frac{\text{age}}{\tau}\right)$ weighting with a 90-day half-life default. **Original cases are preserved** — only search weights are adjusted, maintaining audit traceability.

Adoption 3 — Dialog Recall Memory (Letta): The dialog recall memory stores past operator conversations in DynamoDB so that the Bedrock conversational interface can recall "that issue we discussed last week" across sessions.

Not adopted: LangMem's prompt self-improvement (audit risk — cannot answer "who approved this prompt?").

All adoptions are **opt-in** and do not affect existing behavior when not configured.

5.12 Change Detection and Impact Review

Changes to code, configuration, models, and data sources are detected via two channels: push (git hooks, pipeline state callbacks, ingestion completion events) for immediate detection, and pull (manifest diff, serving metric polling) for periodic detection. When a change is detected, the checklist for affected pipeline parts is re-executed. In the AWS environment, Sonnet reads the diff and reasons about downstream impact, enabling operators to discuss the impact assessment interactively.

6 Regulatory Compliance

6.1 Korean FSS Guidelines Mapping

Table 9 maps the key requirements of Korean FSS AI guidelines to the corresponding system components and verification methods.

6.2 EU AI Act Mapping

Table 10 maps the core EU AI Act provisions to system compliance mechanisms. This paper treats financial recommendation as high-risk AI for compliance-mapping purposes, consistent with Annex III Section 5(b) (creditworthiness assessment and credit scoring), requiring compliance with Title III Chapter 2 obligations.

6.2.1 Compliance Pipeline Implementation

The GDPR/KFCPA compliance obligations listed above are enforced through a pipeline of four modules connected to the Lambda handler and `RecommendationService`:

- **`ConsentManager`**: verifies that the customer has granted AI recommendation consent before any prediction is executed. Absence of consent short-circuits the pipeline and returns a compliant blocked response without logging feature data.
- **`AIDecisionOptOut`**: implements GDPR Article 22 and Korean Personal Information Protection Act (PIPA) right to refuse AI profiling. When a customer's opt-out flag is set, the handler returns a blocked response immediately, before any model inference occurs.
- **`RegulatoryComplianceChecker`**: runs KFCPA §17 suitability assessment before the recommendation is generated. Product risk level is compared against the customer's registered risk tolerance; unsuitable recommendations are blocked with an audit record rather than silently filtered.
- **`ProfilingRightsManager`**: handles data access and deletion requests under GDPR Articles 15 and 17, providing a single entry point for data-subject rights requests that propagates to the feature store and audit tables.
- **`ComplianceAuditStore`**: logs every prediction with the executing task list, serving layers activated, elapsed

FSS Requirement	System Component	Verification	Status
Explainability	Gate weights + 3-agent reason	Per-recommendation audit log	Default
Fairness	Fairness monitor (DI/ SPD/EOD)	Weekly automated report	Default
Model validation	Champion-Challenger (offline + online)	Pre-deployment metric gate + post-deployment traffic gate	Default
Monitoring	Drift detector (PSI)	Continuous, 3-day trigger	Default
Audit trail	HMAC hash-chain logs	Immutable, 7 audit tables	Default
Fallback	Template reason + kill switch	Instant manual override	Default (template) / opt-in (kill switch)
Model risk mgmt	Offline `ModelCompetition` gate + `--force-promote` override	Significance-gated verdict + operator sign-off; audited decision on every registration	Default
Customer suitability	Constraint engine + eligibility filters	Pre-recommendation suitability check	Default

Table 9: Korean FSS guideline compliance mapping. The Status column distinguishes components active in the shipped default configuration (Default) from those requiring explicit activation (opt-in) — a regulator reading this table should be able to tell what runs by default from what merely exists in the codebase.

Article	Requirement	System Component	Status
Art. 13	Transparency	3-agent reason generation + feature attribution	Default
Art. 14	Human oversight	Human-in-the-Loop review + kill switch	Default (review) / opt-in (kill switch)
Art. 15	Accuracy & robustness	Ablation-validated degradation + drift monitoring	Default
Art. 9	Risk management	MRM lifecycle: develop → validate → approve → monitor → retrain	Default
GDPR Art. 22	Right to opt-out	Consent/opt-out audit table + manual override	Default

Table 10: EU AI Act article-level compliance mapping. Status semantics as in Table 9.

time, and compliance check outcomes. The log is appended in real time and is the source of truth for audit trail integrity checks performed by the AuditAgent.

All checks operate under one of two explicit failure postures. In the default fail-open path, if the compliance service is temporarily unavailable, the recommendation is restricted to pre-approved low-risk products only (check cards, demand deposits), the event is recorded at ERROR level with a structured marker (`[COMPLIANCE_DISABLED_UNCHECKED]`) designed to drive a CloudWatch metric-filter alert rather than a mere warning line, and the incident is escalated for human review within the next monitoring cycle. A fail-closed posture is additionally implemented as a configuration option (`compliance.hooks.strict`): when enabled, a hook-construction failure aborts the Lambda cold start, so no request is ever served unchecked; this is the

recommended production posture. The shipped default remains `strict: false`, in which case the compensating control above — restriction to products that do not require suitability assessment, plus alert-grade structured logging — is what prevents a transient infrastructure failure from becoming either a customer-facing outage or a silent compliance bypass.

6.2.2 Extended Regulatory Modules (v2)

The v2 revision of this paper includes an extended set of regulatory modules built on a unified `ComplianceStore` / `SLATracker` foundation. Seven of these hooks (M1/M4/M5/M6/M9/M10/M12) are constructed by a single config-driven entry point, `build_compliance_hooks` (`core/serving/hook_builder.py`), which reads the `compliance.hooks` block of `pipeline.yaml` and returns the complete hook set; both serving entry points (the Lambda handler and the container app) invoke it at startup, so the extended compliance path is wired by default (`compliance.hooks.enabled: true`) rather than assembled by hand per deployment. Each module remains **pluggable**: `RecommendationService` still accepts the hooks as optional constructor injections and defaults to the pre-v1 behaviour when they are absent — but this injection fallback is retained for test isolation and per-environment opt-out, not as the production wiring mechanism.

- **User rights managers** (`core/compliance/rights/`). A three-class suite — `OptOutManager`, `ProfilingWorkflow`, and `ExplanationSLATracker` — covering Korean PIPA §37의2 (opt-out + explanation right), Credit Information Act §36의2 (profiling access / correction / deletion / recomputation), and the statutory 30-day response deadline of PIPA Enforcement Decree §44의3(5) (the tracker enforces a stricter internal 10-day SLA). For §36의2 the suite additionally produces a structured `CREDIT_EXPLANATION` disclosure (`CreditExplanationElements`: whether an automated evaluation took place, its result, the principal criteria applied, and the feature → source lineage of the underlying base data), kept as a separate branch from the PIPA §37의2 general explanation because the two statutes enumerate different disclosure elements. Every request is persisted as a `ComplianceRequest` with an automatic SLA deadline and every disposition emits a `SLA_BREACH` event when the deadline is missed — with the deployment caveat that the shipped default backend is `in_memory` (`pipeline.yaml`); "persisted" becomes durable, retention-grade storage only when the provisioned DynamoDB backend (IaC included) is enabled, which is what multi-year retention expectations such as AI Basic Act §35③ presuppose. The same code-exists-versus-actually-running distinction applies to the human review queue, whose shipped default store is likewise in-memory.
- **Korean AI Basic Act FRIA** (`core/compliance/fria_assessment.py::KoreanFRIAAssessor`). Implements the 7-dimensional assessment enumerated in AI Basic Act Enforcement Decree §27 (data sensitivity, automation level, scope of impact, model complexity, external dependency, fairness risk, explainability gap) with a mandatory 5-year retention period (§35③). This is held as a **separate** class from the EU AI Act Article 9 evaluator to preserve legal-basis separation in audit reports.
- 금감원 **AI RMF classifier** (`core/compliance/ai_risk_classifier.py`). Six-dimensional weighted risk grade (high / medium / low) with explicit detection of grade escalation between successive model versions; an escalation to 'high' requires additional operator approval via the promotion gate.
- **36-item compliance registry** (`core/compliance/compliance_registry.py`). A-group (18 implemented items) plus GAP-group (18 identified gaps, each linked to a concrete Sprint deliverable). Item checks are declarative (`module_exists` / `file_exists` / `config_key` / `custom_check`) so quarterly compliance reports can be regenerated automatically from the live code state.
- **Promotion gate** (`core/evaluation/promotion_gate.py`). Wraps the FRIA + AI Risk evaluations as a post-check on `ModelCompetition`, called from `scripts/submit_pipeline.py::_decide_promotion` per CLAUDE.md §1.10 (single promotion entry point). UNACCEPTABLE FRIA grade or risk escalation to 'high' blocks promotion; the `GateVerdict` is recorded to the audit log. The gate composes three cooperating subsystems:
  - **Dimension score providers** (`core/compliance/dimension_scores.py`). An earliest-wins composite chain: `ManualScoreProvider` (operator overrides keyed by model_version), `MetricsDerivedScoreProvider` (seven `HEURISTIC_RULES` mapping metadata paths → dimension scores via `identity` / `one_minus` / `log10_ratio` transforms, clipped to $[0, 1]$), and a default 0.5 fallback. Each rule records its metadata path, transform, and whether fallback fired, yielding an auditable derivation trail.
  - **MetadataAggregator** (`core/compliance/metadata_aggregator.py`). Six built-in sources feed

the metrics-derived provider: (i) feature lineage, (ii) fairness archive, (iii) human review queue depth, (iv) model registry, (v) LLM configuration slots, (vi) static overrides. Two of the sources (lineage YAML, fairness Parquet) are *archive-backed*, so metadata survives a fresh `submit_pipeline` process start rather than depending on an empty live runtime instance. Results are TTL-cached (default 300 s) to amortise repeated lookups across a single promotion decision.

- **Audit + tracker integration**. `GateVerdict.details` carries the metadata snapshot plus the per-dimension derivation trail (path, raw value, transform, fallback-used flag, final score). `AuditLogger.log_model_promotion(gate_details=...)` embeds this payload in the HMAC-signed, hash-chained record, and `_run_promotion_gate` simultaneously records every non-skip verdict as a `promotion_gate_verdict` artifact in `SageMakerComplianceTracker`. Tracker failures are swallowed — a compliance-tracking outage never blocks a promotion decision, and the missing entry is detectable via the audit log alone. This best-effort posture applies to the tracker only: the audit log itself can be configured fail-closed via `AUDIT_FAIL_CLOSED`, so the two channels deliberately fail in opposite directions.

The gate ships with `compliance.promotion_gate.enabled: true` in `pipeline.yaml` because the provider chain is now wired by default; prior releases kept it disabled to avoid the conservative-LIMITED collapse that occurs when every dimension falls back to 0.5. Operators who still want to disable it per-environment can flip the flag without removing the providers.

- **Human review queue + LLM marker** (`core/serving/review/human_review_queue.py`, `core/recommendation/reason/marker_applier.py`). A three-tier review queue (5% post-hoc sample / 100% mandatory agent / 100% human fallback) covering AI Basic Act §34 human oversight. `MarkerApplier` idempotently appends the AI-generation disclosure mandated by §31 to every L2a LLM-generated reason string.
- **Dynamic item universe loader** (`core/recommendation/universe/dynamic_loader.py`). Reads the candidate campaign + product set from Parquet via DuckDB (local and `s3://` paths), filtered to a configured campaign status set (`approved` / `running`). Prevents the pipeline from serving recommendations for canceled or not-yet-approved campaigns.
- **Audit archive extensions** (`core/recommendation/audit_archiver.py`). Five additional columns — `thinking_trace`, `hallucination_flags`, `tools_used`, `critique_verdict`, `agent_tier` — to support the 5-agent architecture's audit requirements while preserving backward compatibility with v1 Parquet files.

All extended modules share the Sprint 0 foundation primitives (`ComplianceRequest`, `ComplianceEvent`, `SLATracker`) so that an audit query against `ComplianceStore` can reconstruct the full regulatory history of any user, model version, or request type from a single event stream.

0.2.3 Phase 2 Hardening Layer (v2)

Nine additional safeguards were layered on top of the core compliance modules to match the operational maturity of the on-premises reference system. Each addresses a specific oversight, robustness, or effective-challenge requirement.

- **Human-Fallback Layer-4 routing** (`core/recommendation/fallback_router.py`). The original 3-layer fallback router (distilled LGBM → direct LGBM → rule engine) gains an explicit fourth layer that routes hopeless cases to a human review queue rather than silently degrading. Activated via `serving.review.tier_3_human_fallback` and surfaced via `route_all()` layer counts.
- **L2a Safety Gate 3-layer** (`core/recommendation/reason/l2a_safety_gate.py`). Every LLM-generated L2a output now passes through three independent checks: (1) structural parsing (length, sentence count), (2) rule-based screening (banned phrases, regex, Korean resident registration + card PII patterns), (3) optional LLM quality score. Any veto forces fallback to the L1 template output; the `SafetyVerdict` records which layer failed for post-hoc analysis.
- 금
  소법 **§17 suitability filter** (`core/recommendation/constraint_engine.py::SuitabilityFilter`). Registered as a constraint-engine filter so it composes with the existing fatigue, eligibility, and owned-product filters. Enforces risk_tolerance ≥ risk_level with additional hard caps for senior customers (≥ 65, max risk 2) and low-income customers (< 30M KRW annual, max risk 3).
- **Counterfactual Champion-Challenger** (`core/evaluation/counterfactual_cc.py`). Off-policy IPS and SNIPS estimators with paired-bootstrap confidence intervals. Supports EU AI Act Art. 15 (accu-

racy) and SR 11-7 effective-challenge by providing evidence that a challenger would have produced better outcomes than the champion on logged observational data.

- **auto-promote=False enforcement** (`core/evaluation/model_competition.py`). The code-level `CompetitionConfig` default is retained at `True` purely for backwards compatibility with legacy test fixtures; production posture is enforced in `pipeline.yaml::serving.competition.auto_promote: false`, which takes precedence at run time. Under this posture a challenger that beats all metric thresholds is still NOT promoted: the operator must re-run with `--force-promote`, satisfying EU AI Act Art. 14 (human oversight) and SR 11-7 model-risk management, and leaving an explicit operator signature in the audit log. Flipping the code default is deliberately avoided so that unit tests using bare `CompetitionConfig()` continue to pass.
- **Annex IV technical-documentation mapper** (`core/compliance/annex_iv_mapper.py`). A dedicated mapper that, for each of the 12 Annex IV sections, resolves a list of evidence sources (Python modules, documents, config keys) and reports which sections have full, partial, or missing evidence. The coverage rate is tracked automatically so the technical file can be regenerated before every conformity assessment.
- **Fairness metrics archive and PromotionGate wiring** (`core/monitoring/fairness_monitor.py`, `containers/lambda/fairness_evaluation.py`). The monitor appends every measurement to an in-memory history and, when `monitoring.fairness.archive_parquet_path` is configured, also to an S3 Parquet file that the governance reporter can query directly. `get_archive(attribute, limit)` supports drill-down dashboards without re-running the computation. Critically, the fairness Lambda now invokes `monitor.archive_metrics()` on every evaluation so the archive actually fills over time. This closes a prior gap where `MetadataAggregator.fairness_archive_parquet_path` pointed at an archive no producer was writing to, causing the PromotionGate `fairness_risk` dimension to silently collapse to its heuristic 0.5 fallback and the gate to converge to a conservative `LIMITED` verdict independent of the challenger. With the producer wired, `MetadataAggregator` now reads real fairness evidence and the gate verdict reflects true challenger behaviour.
- **Drift persistence + markdown report** (`core/monitoring/drift_detector.py`). Each `detect_drift()` call optionally writes one row per feature to a Parquet archive, preserving PSI scores, severity labels, and the thresholds that were active at evaluation time. A companion `generate_markdown_report()` produces a human-readable summary suitable for the weekly governance bundle. The archive writer is S3-aware (a dedicated `parquet_io` helper handles both local and `s3://` paths), and the drift → auto-retrain chain was re-verified end-to-end after fixing a bug in which a 2-D ndarray passed to the PSI check raised a swallowed `TypeError`, leaving the retrain trigger permanently `False`.
- **Lineage catalog extensions** (`core/monitoring/lineage_tracker.py`). The lineage tracker now accepts runtime registrations (`register_feature_mapping`), loads YAML-defined catalogs (`load_mapping_from_yaml`), and returns coverage reports (`coverage_report`) that quantify how many of the current feature set have a source-table mapping. Required for AI Basic Act §34 (training-data provenance) when the feature count grows.

The `test_phase2_should.py` suite exercises all nine modules (36 tests, all passing) and verifies that `pipeline.yaml` as shipped forces `auto_promote: false`.

6.2.4 Learning-Stack & Interpretation Layer (v2)

Four additional modules extend the hardening layer into the learning stack (feature selection, evidential head, temporal ensemble) and the reason-generation context, closing the remaining Paper 2 v2 evidence gaps for real-data safety and multidisciplinary interpretation.

- **IG 3-stage feature selection** (`core/training/feature_selector.py::select`). The final `select()` call now runs Stage 1 (IG cumulative-importance), Stage 2 (LGBM-gain pruning), and Stage 3 (mandatory feature guarantee — domain-critical features are restored to the selection even if Stages 1-2 dropped them). Stage 3 logs a warning for mandatory features not present in the current feature schema and records the restored features in `FeatureSelectionResult.mandatory_included`.
- **Evidential `valid_mask` missing-data guard** (`core/model/layers/evidential.py`). The evidential head now accepts an explicit `valid_mask: Tensor[B]` and, in its absence, auto-detects non-finite rows via `torch.isfinite`. Invalid rows receive a task-type-specific neutral prediction (0.5 for binary, 1/K for multiclass, 0.0 for regression) plus the canonical max-uncertainty value. `NaN`/`Inf` gradients are prevented by running the linear layer on a `nan_to_num`-scrubbed copy. The effective mask is returned in the evidence-

info dict so downstream losses can exclude invalid rows.

- **HMM-smoothed ensemble gating** (`core/model/experts/temporal.py::set_hmm_routing`). The `TemporalEnsembleExpert` exposes a runtime `set_hmm_routing(enabled, smoothing, transition_prior)` method and an equivalent `hmm_routing` config block. When enabled, gating weights produced by the learned gate are post-multiplied by a row-stochastic transition matrix and renormalised, acting as a 1-step HMM forward smoothing that reduces per-sample variance of the gating decision. The transition matrix is held as a non-persistent buffer so it moves with `.to(device)` but does not train.
- **Multidisciplinary interpreter hook** (`core/recommendation/reason/context_assembler.py`). The `ContextAssembler` now accepts an optional `multidisciplinary_interpreter` argument — either an object with an `interpret(context_dict) -> dict` method or a plain callable with the same signature. Each `assemble()` call invokes the interpreter and attaches its output to `AssembledContext.multidisciplinary_insights`. Failures are swallowed so a buggy interpreter cannot break reason generation; the attached interpreter can be swapped at runtime via `attach_interpreter(...)`.

All four modules remain backward-compatible: legacy callers that do not pass the new arguments preserve the pre-v2 behaviour exactly. `tests/test_phase2_remaining.py` (23 tests, all passing) exercises each safeguard with both the legacy and enhanced code paths.

6.2.5 SageMaker-Native Compliance Tracking (v2)

The on-premises reference system uses MLflow for experiment tracking and DVC for dataset versioning, the de-facto open-source combination. AWS provides managed equivalents for every role MLflow + DVC played, so rather than lifting the OSS stack into the cloud we wrap the native services behind a regulation-aware tracker:

- **MLflow tracking → SageMaker Experiments** (Trials / TrialComponents)
- **MLflow model registry → SageMaker Model Registry** (already handled by `core/serving/model_registry.py`)
- **MLflow lineage → SageMaker Lineage** + the feature-to-table `DataLineageTracker`
- **DVC dataset versioning → S3 versioning** (plus the `artifact_s3_prefix` config key that points the tracker at the bucket)
- **Managed MLflow (2024) → SageMaker Managed MLflow** (picked up automatically by switching the backend to `sagemaker` in `compliance.tracking`)

`core/compliance/sagemaker_compliance_tracker.py` implements this wrap:

- Backend abstraction (`ComplianceTrackerBackend`) with two concrete implementations: `InMemoryTrackerBackend` for tests and local dev, `SageMakerTrackerBackend` for production. Both expose the same `put_artifact` / `list_artifacts` surface so callers are backend-agnostic.
- Five artifact types cover the Sprint 0 4 regulatory surface: `fria_assessment`, `ai_risk_assessment`, `compliance_registry_sweep`, `promotion_gate_verdict`, `custom`. Each type maps to a dedicated `log_*` helper that extracts the relevant parameters and metrics from the domain dataclass and attaches artifact-type + grade / risk-category tags so the SageMaker console (and any Athena export) can filter by regulatory concern.
- TrialComponent names are capped at the SageMaker 120-char limit; failed boto3 calls are logged and treated as best-effort so a tracking outage never blocks a regulatory decision.
- The backend is selected via `pipeline.yaml::compliance.tracking.backend`; the shipped default is `sagemaker` (production IAM has been verified reachable — `list_experiments` succeeds, and `describe_experiment` returns `ResourceNotFound` for an unseeded name rather than an access-denied error, confirming the training-job role carries the required Experiments permissions). The `in_memory` backend is retained for unit tests and local development that must remain hermetic, and is opted into by overriding the flag.

`tests/test_sagemaker_compliance_tracker.py` (24 tests) exercises both backends with a synthetic boto3 client stub that records each `create_experiment` / `create_trial_component` / `list_trial_components` call so we can assert on the exact request shape (required fields, artifact-type tag presence, risk-category tag presence, 120-char name cap, `Completed` status, etc.) without making a real AWS call.

This closes the Paper 2 v2 evidence gap for **regulatory artifact versioning** without depending on the MLflow / DVC stack, keeping the AWS deployment aligned with the “ cloud extension of on-prem, not

a separate system " positioning documented in `docs/pipeline_comparison_matrix.md`.

6.2.6 DuckDB-over-Parquet compliance SQL (v2)

The on-premises reference system uses DuckDB as the SQL engine over Parquet audit archives; AWS's paid equivalents (Athena, Redshift Spectrum, S3 Select) introduce infrastructure cost and operational overhead that the observed query volume does not justify. Instead, `core/compliance/audit_sql.py::ComplianceSQLHelper` wraps DuckDB's httpfs extension so `s3://` audit archives can be queried with the same SQL the on-prem deployment already uses — at zero added infrastructure cost.

- `register_view(name, parquet_uri)` attaches a Parquet glob / directory / `s3://` URI as a named DuckDB view. Views are re-creatable so new batches appended to the same prefix are picked up on demand.
- `query(sql, params)` runs parameterised SQL and returns row dicts.
- Four regulator-focused convenience methods — `recent_opt_outs`, `consent_changes_for_user`, `sla_breaches`, `promotion_gate_history` — take a `since` argument defaulting to the configured 30-day window, covering the typical quarterly-audit request shape.
- `counts_by_column(view, column)` produces `{value: count}` summaries for dashboards.
- `pipeline.yaml::compliance.audit_sql.paths` auto-registers views at construction time so the helper is ready to use in one line.

Because views are registered against any combination of `s3://` and local paths, an auditor running the same SQL against an `InMemoryComplianceStore`-backed test fixture and a production `S3ParquetComplianceStore` export gets identical results. `tests/test_compliance_sql.py` (22 cases) verifies the config surface, view lifecycle, parameterised queries, each convenience method, and a cross-view JOIN across `opt_out` and `consent` views to confirm that multi-table analytical queries work without any additional infrastructure.

`docs/pipeline_comparison_matrix.md` records the design-choice rationale alongside the other paid alternatives considered.

6.2.7 Pearl Ladder Completion & LLM Hardening (v2)

Paper 2 v1 claimed coverage of Pearl's causal ladder at two rungs — Rung 1 (observation) via the Causal Explainability Head and Rung 3 (counterfactual) via the CCP + Counterfactual Champion-Challenger. v2 fills the Rung 2 (intervention) gap with an uplift-learner module and adds a second LLM hardening layer that complements the existing `PromptSanitizer`.

Rung 2 uplift learners (`core/evaluation/uplift_learner.py`). Both T-Learner and X-Learner implementations are shipped with a matching numpy-only fallback regressor and a logistic propensity estimator. T-Learner fits independent outcome models for the treated and control populations and returns tau(x) = mu_1(x) - mu_0(x); X-Learner adds a cross-fit stage that reweights imputed residuals by propensity, yielding more stable CATE under treatment-group imbalance. Evaluation uses the Qini coefficient (area under the Qini curve, normalised by the perfect-targeting ideal) and the top-K-percent uplift metric (mean observed effect within the predicted-top fraction). Both learners expose a fit(X, T, Y) / predict(X) interface so any sklearn or LightGBM regressor can be swapped in as the stage-1 model without changing the evaluation pipeline. The numpy-only implementation means offline evaluation, unit tests, and CI all run without heavy dependencies.

Regulatory positioning. Rung 2 is where the recommender can justify treating / not-treating an individual customer based on the estimated heterogeneous effect, which AI Basic Act §35 (FRIA) and PIPA §37의2 (automated decision rights) both scrutinise. The uplift estimator output is audit-logged as a regulatory artifact via the SageMaker compliance tracker when the gate is enabled, so the rationale for choosing to target a given segment is reproducible.

LLM hardening — AI security checker (`core/security/ai_security_checker.py`). The existing `PromptSanitizer` handles sensitivity classification, PII scrubbing, and VPC routing; the security checker adds two complementary layers. On the request side, fourteen default prompt-injection / jailbreak patterns (ignore previous instructions, DAN mode, reveal system prompt, tool-call breakouts, base64-encoded payloads) are scanned before the prompt is sent to any LLM provider. On the response side, eight output-leak patterns catch system-prompt echoes, meta-instruction leaks, role-breakout phrasing, and Korean-resident-registration / card-number PII. Findings return a `SecurityVerdict` with per-finding severity; `wrap_provider` produces a drop-in replacement for any LLM provider that rewrites a blocked exchange to a safe refusal (configurable callbacks).

Combined with the Sprint 2 L2a Safety Gate (`core/recommendation/reason/l2a_safety_gate.py`), the LLM path now has four independent hardening layers — sensitivity-based routing, rule-based safety gate, AI security checker (prompt + output), and the LLM generation marker — each of which can veto independently and

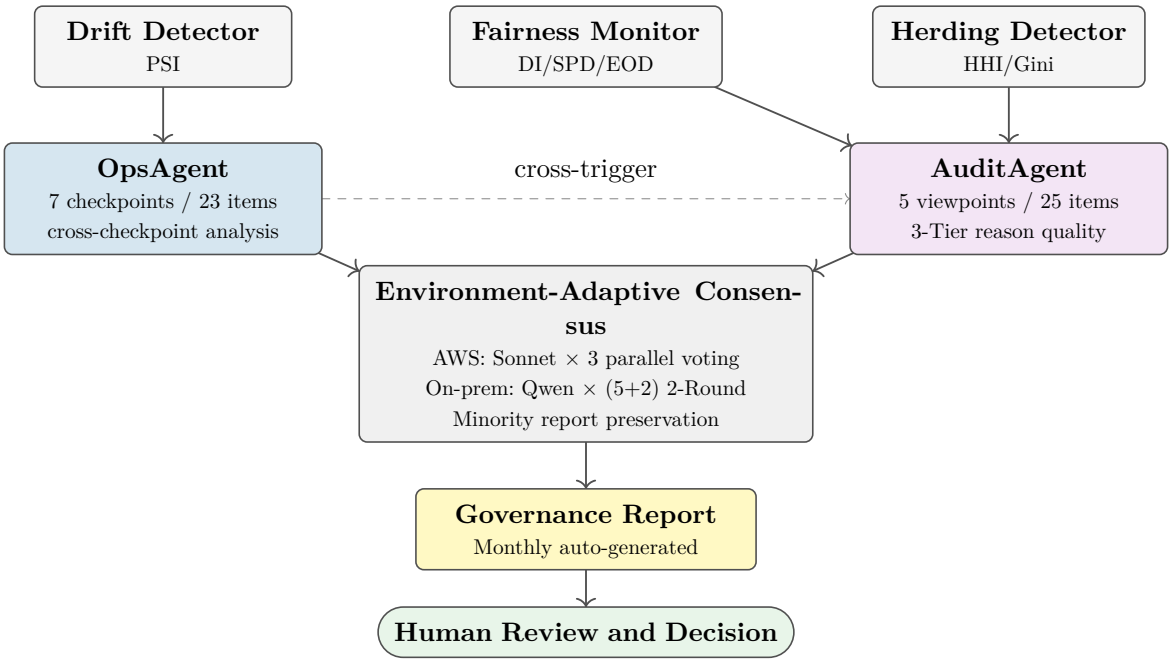

Figure 7: Monitoring and governance architecture.
Monitoring components → Ops/Audit agents → Consensus → Governance report → Human review.

each of which is config-driven so a security ops team can extend the rule catalogue without a code change.

`tests/test_phase3_could.py` (39 cases) covers T-Learner / X-Learner CATE recovery, qini plus top-K-percent uplift metrics on synthetic uplift data, every default security pattern, the provider-wrapping end-to-end flow, and custom refusal callback injection. Together with the Phase 1 and Phase 2 suites, plus the PromotionGate MetadataAggregator live-wiring tests and the post-normalization feature-group-range rebuild regression suite, the AWS-side regulatory, learning, and security stack is exercised by *785 tests (785 passing)*.

6.3 Korean AI Basic Act

Korea's AI Basic Act (passed by the National Assembly December 2024, promulgated January 2025, effective January 22, 2026) [13] introduces a domestic high-risk AI classification framework. Financial product recommendation falls within the high-risk category, requiring impact assessment, transparency obligations, and human oversight. Our system's existing compliance architecture (drift monitoring, fairness auditing, audit trails, and human-in-the-loop review) aligns with the Act's requirements, with the governance reporting module generating documentation suitable for regulatory submission.

6.3.1 Deployment Scope Restriction

Low-risk products only. The actual deployment target of this research is **low-risk products** such as check cards, deposit accounts, and demand deposits. Investment products (funds, stocks, bonds) and insurance products are **intentionally excluded** from the deployment scope, because the Korean Financial Consumer Protection Act Article 17 suitability principle, EU MiFID II, US Reg BI, and similar frameworks in many jurisdictions restrict direct AI recommendations of such products or mandate human advisor oversight due to mis-selling risk.

Synthetic benchmark data (Santander 941K) includes product-agnostic tasks for benchmarking purposes — including tasks like `will_acquire_investments` — but **operational deployment is limited to check card recommendations**. This separation of "what the model learns" from "what the system actually recommends" preserves the generality of benchmark evaluation while minimizing regulatory risk.

Why this distinction matters. If a reviewer sees Paper 2 examples ("Why was this card recommended to me?") and asks "why are investment products not shown?", the answer is **intentional exclusion**. The underlying technology — AI agents generating natural-language recommendation reasons — can technically be applied to investment products, but fully eliminating mis-selling risk requires human advisor oversight, which is incompatible with the **automation** claim of this paper. By restricting to low-risk products, the claim of "fully automated, explainable recommendations" becomes compatible with regulation.

6.4 Monitoring and Governance

6.4.1 Human-in-the-Loop

Regulatory bodies (Korean FSS, EU AI Act Art. 14) require human oversight. The system implements this at multiple levels:

- **Reason sampling review**: Periodic human review of generated reasons.
- **Model replacement approval**: The offline Champion-Challenger gate computes the promotion verdict; under the shipped `auto_promote: false` posture, promotion additionally requires explicit operator sign-off (`--force-promote`), so no model reaches production on statistical significance alone. Every decision is recorded to an HMAC-signed, hash-chained audit log.
- **Incident escalation**: Automated anomaly detection triggers human investigation.
- **Fairness review**: Periodic human audit of fairness metrics.

The kill switch referenced in the mapping tables is implemented with fail-closed semantics: serving consults a dedicated DynamoDB table (`ple-kill-switch`, provisioned via IaC), and a read failure is treated as ACTIVE — when the switch state cannot be established, the governed path halts rather than assuming permission. The on-premises counterpart is available as an opt-in injection, disabled by default and scoped to next-best-action recommendations only.

6.4.2 Monitoring Implementation

The monitoring layer is composed of three purpose-built modules that feed the governance pipeline:

- **`DriftDetector`**: computes Population Stability Index (PSI) per feature between successive distillation runs. Feature-level PSI complements the existing prediction-level drift signal, enabling early detection of input distribution shifts before they degrade model performance.
- **`FairnessMonitor`**: evaluates demographic bias across protected attributes (age group, gender, income tier) in batch serving predictions. Outputs disparate impact (DI), statistical parity difference (SPD), and equalized odds difference (EOD) per task, providing the per-segment audit trail required by EU AI Act Art. 10(2)(f).
- **`GovernanceReportGenerator`**: produces a per-distillation-cycle governance report consolidating drift status, fairness findings, audit trail integrity, and checklist compliance. Reports are saved to S3 with HMAC signatures, forming the documentation corpus for regulatory submission.

`AuditLogger` records the start and completion of training and distillation events as structured log entries, creating the time-ordered provenance chain that links every deployed model version to its training run and input data snapshot.

6.5 Pipeline Auditability for High-Risk AI

The preceding sections map individual system components to regulatory articles. However, the EU AI Act's high-risk AI requirements (Title III, Chapter 2) demand auditability of the *entire ML pipeline* — not just the serving layer. This section demonstrates how our training infrastructure, described in the companion paper, satisfies these requirements end-to-end.

6.5.1 Data Governance (Art. 10)

Article 10 requires that training data be "relevant, representative, free of errors and complete" with appropriate data governance measures. Our Phase 0 pipeline (preprocessing → feature generation → label derivation → normalization → tensor storage) produces two audit artifacts at every run: The feature statistics file records per-column NaN ratios, zero-variance flags, distribution statistics, and generated feature counts; the label statistics file records class balance and positive rates for all 13 tasks. These artifacts constitute a verifiable data quality record that an external auditor can inspect without executing any code.

Reproducibility is guaranteed by config-driven processing: the entire pipeline is controlled by a split-config of three YAML configuration files (the pipeline configuration, the feature group configuration, and the dataset-specific task/label configuration), so identical configs produce identical outputs given the same input data. No dataset-specific logic resides in executable code.

As a pre-training gate, the leakage validator verifies that (1) the scaler was fit on training data only, (2) a temporal gap separates train/validation/test splits, and (3) the final sequence timestep does not overlap with label windows. Training is blocked if any check fails — ensuring data governance violations are caught *before* compute is consumed.

6.5.2 Technical Documentation (Art. 11)

Article 11 mandates technical documentation sufficient to assess compliance. Our system achieves this through config-as-documentation: the three YAML configuration files fully specify feature groups, generator parameters, normalization stages, task definitions, loss weights, and expert routing. A config snapshot is saved alongside every training run, and the evaluation metrics file cap-

tures the full training provenance (hyperparameters, data splits, final metrics, timestamps). Combined with fixed random seeds, any historical experiment can be exactly reproduced from its archived config.

6.5.3 Logging and Traceability (Art. 12)

Article 12 requires automatic recording of events for the identification of risks and substantial modifications. The training loop produces per-epoch logs with task-level loss breakdowns, enabling auditors to trace exactly when and where performance changed. NaN/Inf detection identifies *which task* produced anomalous gradients, rather than reporting a generic training failure. Gradient norm monitoring flags when norms exceed the clipping threshold by an order of magnitude, providing a training stability audit trail. Checkpoints are versioned with their corresponding eval metrics, creating a complete lineage from data to deployed model.

6.5.4 Human Oversight (Art. 14)

Beyond the Human-in-the-Loop mechanisms described above, the system provides two additional transparency layers for human reviewers:

- **Post-distillation LGBM students** expose gain-based and SHAP feature importance rankings per task — a human-readable audit that compliance officers can review without deep learning expertise.
- **CGC gate weights in the PLE teacher** quantify each expert's contribution to every prediction, making the expert routing mechanism transparent rather than a black-box ensemble.

The 3-agent LLM pipeline (Section 4) further translates these technical attributions into natural-language explanations that compliance officers can directly evaluate.

6.5.5 Accuracy and Robustness (Art. 15)

Article 15 requires that high-risk AI systems achieve "an appropriate level of accuracy, robustness and cybersecurity." The companion paper's ablation study demonstrates *graceful degradation*: removing any single expert reduces performance incrementally rather than causing catastrophic failure, proving that no single component is a critical point of failure. Uncertainty weighting [11] prevents any single task from dominating the multi-task objective, ensuring robust performance across all 13 tasks simultaneously. Drift monitoring (PSI-based) is designed to provide continuous robustness verification once deployed.

6.5.6 Bias Monitoring (Art. 10.2f)

Article 10(2)(f) requires examination of training data for possible biases that may affect health, safety, or fundamental rights. Label distribution validation before training (via the label statistics file) ensures class imbalance is documented and addressed. The task-level breakdown in the evaluation metrics file enables per-segment performance monitoring: auditors can verify that model accuracy does not vary systematically across customer demographics or product categories. The fairness monitor component (disparate impact, statistical parity difference, equalized odds difference) provides automated bias detection at the granularity required by Article 10(2)(f).

6.5.7 Model Risk Management Lifecycle

The system implements an SR 11-7 aligned five-stage MRM lifecycle: *develop* (teacher training + student distillation), *validate* (independent metric evaluation on held-out data), *approve* (two-stage gate, see below), *monitor* (continuous PSI-based drift detection), and *retrain* (automatic re-distillation when drift exceeds threshold). The *approve* stage is implemented as a two-gate pipeline. The **offline gate** (`scripts/submit_pipeline.py::_decide_promotion`) runs immediately after distillation and applies a single short-circuit decision ladder in a fixed order, so every challenger is evaluated against the same sequence and the outcome is fully auditable: (1) if `--force-promote` is set, the challenger is promoted unconditionally as an operator override (`trigger=manual`); (2) otherwise, if no champion exists in the registry, the challenger is bootstrap-promoted and no comparison is attempted; (3) otherwise, if `fidelity_summary.failed > 0`, the challenger is rejected by a safety floor regardless of its training metrics and Competition is skipped, preserving the teacher-student fidelity guarantee; (4) otherwise, `ModelCompetition.evaluate` compares the challenger against the current champion's recorded metrics, requiring the primary metric to improve by at least `min_improvement` (default 0.5%) with no secondary metric degrading by more than `max_degradation` (default 2%), and optionally a paired-bootstrap significance test, and returns a promote / reject verdict. Every outcome — `bootstrap`, `promote`, `reject`, or `force_promote` — is recorded by `AuditLogger.log_model_promotion` to an HMAC-signed, hash-chained S3 WORM log, producing a non-repudiable audit trail of who promoted what and why. The **online gate** (`ModelMonitor.evaluate_champion_challenger`) is invoked only after step (4) has promoted a challenger and sufficient DynamoDB prediction records have accumulated to apply a two-proportion z-test on realized traffic; it is scaffolded but not yet scheduler-wired, so it is deliberately **not** part of the automatic offline ladder above and activates only once real traffic volume justifies the comparison.

6.5.8 Per-Prediction Attribution Audit (CEH Integration)

Model-promotion events cover **which model** is in production. A distinct requirement, emphasized by GDPR Art. 22 ("right to a meaningful explanation of automated decisions") and the EU AI Act Art. 13 (transparency to affected persons), is audit coverage of **individual decisions**: if a customer asks "why did the model recommend this, and why at this time?", the system must be able to reconstruct the feature attributions that drove that specific prediction, unaltered, from a tamper-evident record.

The Causal Explainability Head (CEH) introduced in the companion loss-dynamics paper (Paper 3) produces a per-prediction attribution vector on every inference. `AuditLogger` exposes `log_attribution(model_id, sample_id, top_features, attribution_hash, input_dim)`, which records for each prediction: (i) the top-K most influential features with their signed weights (a compact summary for human review), (ii) a SHA256 hash of the complete float32 attribution vector (for cryptographic replay verification), and (iii) the same HMAC signature

1. hash-chain linkage used for promotion events. The audit record is

therefore:

$$\text{entry}_i = (\text{timestamp}, \text{op}, \text{sample\_id}, \text{top\_K features}, \text{hash}(\text{attr}_i), \text{prev\_hash}, \text{HMAC})$$

An auditor can reproduce the decision by re-running inference on the archived input and recomputing the attribution vector hash; matching the stored hash proves the record is authentic; mismatching hashes prove tampering. The hash-chain verifier (`verify_chain`) detects insertion, deletion, or modification of any intermediate record, so selective suppression of inconvenient decisions is detectable even by adversaries with write access to the storage medium. Strengthening this over v1, `verify_chain` now additionally re-verifies each entry's HMAC signature with a constant-time comparison: an adversary who rewrites a record and recomputes the downstream hash links still fails per-entry signature re-verification without possession of the signing key, closing the tamper-and-relink path that hash-chain checking alone leaves open.

6.5.8.1 Signing-Key Governance

A signature is only as trustworthy as the key behind it, so the v2 audit stack treats key management as part of the audit design rather than a deployment detail. The logger refuses to start in `prod` or `staging` environments with the publicly known default key — construction raises `AuditIntegrityError` (`core/monitoring/audit_logger.py`) — eliminating the most common HMAC deployment failure: signing with a key the adversary can read in the repository. Production keys are injected from SSM Parameter Store (provisioned by `setup_aws.sh`) rather than stored in configuration files. When `AUDIT_FAIL_CLOSED` is set, a failed S3 write of an audit record propagates as an exception instead of being swallowed, so the system cannot unknowingly operate with auditing silently broken; and the audit bucket itself is provisioned with S3 Object Lock in COMPLIANCE mode via IaC, making the chain append-only even for credentialed administrators. The on-premises reference system enforces the same posture (default-key refusal, fail-closed write option), keeping the two environments symmetric.

The integration is intentionally minimal: CEH exposes a public accessor (`get_last_attribution`) on the causal expert, the PLE model surfaces it via `get_ceh_attribution`, and the serving path pairs each prediction with one `log_attribution` event. No model surgery; the attribution vector is a by-product of the training-time regulariser described in the companion loss-dynamics paper.

6.5.9 Causal Guardrail Audit (CG Integration)

CEH answers "why did the model recommend this?"; the Causal Guardrail answers the adjacent question "can we trust this recommendation?". In the companion loss-dynamics paper's causal-guardrail findings we established that a z-space Mahalanobis score on the causal expert's latent detects three synthetic OOD probe types at 100% TPR with 5% FPR — a regulator-usable reliability signal at no training cost. The audit path mirrors the attribution path.

`AuditLogger.log_guardrail(model_id, sample_id, coherence_score, threshold, triggered)` records, for each evaluated prediction: the score, the threshold in force at that time, and a boolean flag for whether the guardrail fired. Combined with the CEH attribution record, an auditor now has a complete per-prediction trail: explanation (what drove this prediction) and reliability (should this prediction be acted on at all). Both records sit in the same HMAC-signed hash-chained store, so tampering with either is detectable under the same verification path.

Operationally, a `CausalGuardrail` helper (`core/monitoring/causal_guardrail.py`) encapsulates the calibration and per-sample check. A caller computes a reference $\boldsymbol{\mu}, \boldsymbol{\sigma}$ batch once (e.g., at deployment, from a held-out in-distribution sample), stores it, and then emits one `log_guardrail` event per prediction. Threshold drift is handled by periodic re-calibration; the stored threshold

Metric	Description	Result
L1 template coverage	Tasks with template-based reason generation	100% (13/13)
Template variants	Distinct templates (6 categories × 5 variants)	30
PII detection patterns	Regex-based PII check rules	5
Compliance rules applied	Suitability, consent, opt-out, profiling, disclosure	5

Table 11: Automated reason quality metrics. Human evaluation is planned for production deployment (pending Bedrock L2a rollout).

Metric	Definition	Result
Grounding score	Fraction of tasks with matched Korean keywords in reason text	0.33 (1/3 sampled)
Readability	Fluency score (no broken template markers)	1.00
Overall quality	$0.4 \times$ grounding $+0.3 \times$ readability $+0.3 \times$ compliance (all normalized to $[0, 1]$)	0.74
Bias DI	Disparate Impact across all protected groups	1.0 (no bias detected)
Domestic compliance (KFCPA)	Rules checked (suitability, consent, opt-out, profiling, disclosure)	5 checked, 0 violations

Table 12: AuditAgent automated reason quality assessment. Grounding score 0.33 = 1/3 sampled tasks had verifiable keyword matches; readability 1.00 confirms no template rendering failures.

field in each audit record lets auditors reconstruct exactly which threshold was in force at decision time. When the drift detector fires on consecutive monitoring windows (configurable threshold, default 3 consecutive days), the retrain stage is triggered automatically, producing a new challenger that re-enters the competition cycle. This closed loop ensures that model risk is managed continuously rather than at discrete annual review points, satisfying both SR 11-7 expectations and EU AI Act Art. 9 risk management requirements.

7 Experiments

7.1 Distillation Experiments

Binary tasks achieved AUC gaps of 0.018–0.036 (mean 2.6 percentage points), with ranking correlation above 0.96 across all 7 tasks. These figures are from the v13 run; the v14 re-run, after the two distillation fixes analysed in the Discussion, reduces the mean binary AUC gap to 0.58 pp, and the fidelity gate now reports its verdict in two tiers (Tier 1 blocking / operational, Tier 2 informational / architecture-inherent) as detailed there. The primary failure mode was calibration gap (0.08–0.10), addressed by post-hoc Platt scaling for probability-critical tasks. Two multiclass tasks (nba_primary, segment_prediction) fell below the 2× random teacher threshold and were routed to direct hard-label training; next_mcc (50-class) was routed to SKIP (Layer 3 rule engine). One regression task (mcc_diversity_trend) achieved an MAE gap of 0.025 via direct training. The other two (product_stability, cross_sell_count) were routed to SKIP due to near-zero teacher R^2.

7.2 Reason Generation Quality

7.2.1 Automated Reason Quality Metrics

Human evaluation is planned for production deployment; automated compliance validation results are reported here. The evaluation covers L1 template coverage, template variant diversity, PII detection, and regulatory rule application.

7.2.2 AuditAgent Reason Quality Assessment

The AuditAgent evaluated recommendation reasons produced by the L1 template engine and L2a Bedrock rewrite pipeline, reporting four automated quality dimensions.

7.2.3 L1 Template Reason Examples

Table 13 shows representative L1 template reasons generated by the production Lambda (ple-predict) for three tasks. These are the verbatim outputs from the TemplateEngine before L2a Bedrock rewrite.

7.2.4 L2a Bedrock Rewrite Example

L2a (Bedrock Sonnet) rewrites the L1 template into a fluent, customer-facing sentence. Below is the measured before/after pair for the `top_mcc_shift` task:

Task	L1 Template Reason
top_mcc_shift	Benefits tailored to your changing spending patterns. We recommend customized spending benefits for your diverse category usage.(를)
will_acquire_investments†	This may align with your financial goals. This investment product is suited to your current financial lifecycle stage.
churn_signal	We aim to maintain your valued transaction relationship. We analyze your usage patterns to recommend a customer retention program.(를)

Table 13: L1 template reason examples from production Lambda (ple-predict), translated from Korean, before L2a rewrite. The artifact "(를)" in rows 1 and 3 is the type of grammatical defect L2a corrects (a Korean object-marker placeholder left by the template engine). †Benchmark-only task; deployment restricted to low-risk products (Section 6.3.1).

Metric	Description	Value
PII pattern count	Regex-based PII detection rules	5
Validation categories	Hallucination, Regulatory, Appropriateness, Tone, Factual (see Table 4)	5
Human review sample rate	Fraction of L2a outputs reviewed by humans	5%
Fallback rate to L1	Configurable threshold; triggered on gate failure	configurable

Table 14: Safety Gate automated metrics. Precision/recall evaluation pending live deployment; validation categories are defined in Table 4.

Component	Latency	Cost/1K req	Notes
L1 predict + 13 tasks (warm)	120ms	$< $0.02	Lambda; measured
L2a first call (Bedrock Sonnet)	2.4s	$0.01	On-demand; cached thereafter
L2a cache hit	6ms	$< $0.001	DynamoDB; measured
Cold start	6s	—	S3 model download; amortized
LanceDB case search	< 100ms	< $0.01	Cold-start grounding; past recommendation cases

Table 15: Serving latency breakdown on warm Lambda (serverless, no GPU). LanceDB grounding uses accumulated recommendation cases per customer.

L1 (template): "Benefits tailored to your changing spending patterns. We recommend customized spending benefits for your diverse category usage.(를)"

Note: "(를)" is a Korean object-marker placeholder artifact left by the template engine; L2a eliminates it.

L2a (Bedrock rewrite): "We recommend a customized spending benefit product so you can enjoy practical benefits across multiple categories in line with your diverse spending patterns."

The L2a rewrite eliminates the template artifact "(를)" (a Korean object-marker placeholder), merges the two-sentence structure into a single coherent utterance, and preserves the grounding signal (spending patterns, categories, benefits). First-call latency is 2.4s (Bedrock Sonnet on-demand); subsequent calls for the same (task, feature-signature) pair retrieve the cached result from DynamoDB in 6ms.

7.3 Safety Gate Evaluation

Table 14 reports the Safety Gate's automated validation metrics, measuring PII detection coverage, regulatory check breadth, and human review sampling rate. These metrics characterize the gate's *coverage*; precision/recall evaluation is pending live deployment with production traffic.

7.4 Serving Performance

Of the 13 tasks served by the production Lambda: 10 tasks are served by Layer 2 LGBM (7 tasks from distillation + 3 from direct hard-label training) and 3 tasks are served by the Layer 3 rule engine (next_mcc, product_stability, cross_sell_count). The DynamoDB audit trail records every prediction event: as of the

measurement date the `ple-prediction-log` table holds 51 items and `ple-audit-distillation` holds 18 items, providing a verifiable history of serving decisions for regulatory inspection.

The entire teacher training and distillation cycle runs on SageMaker Spot instances, with GPU (ml.g4dn.xlarge) for teacher training and CPU (ml.m5.4xlarge) for LGBM distillation; distillation was moved up from ml.m5.2xlarge after the 13-task soft-label workload exceeded its memory, and a representative spot run completed in 62 minutes at $0.30. Spot pricing reduces compute cost by approximately 70% versus on-demand; interruption risk is managed by automatic checkpointing and job resume.

7.5 Regulatory Compliance Audit

We evaluate regulatory alignment across three dimensions: *checklist compliance*, *audit trail integrity*, and *fairness metrics*.

Checklist compliance. The system implements 14 regulatory requirements mapped from the Korean Financial Consumer Protection Act (금소법, KFCPA) Articles 17–19, EU AI Act Articles 13–14, and FSS AI Guidelines. Key items include: suitability assessment before recommendation (KFCPA Art. 17), AI-generated content disclosure (AI Basic Act §31), human oversight mechanism (EU AI Act Art. 14), and opt-out functionality.

Audit trail integrity. Every recommendation reason is logged with: (1) the LGBM SHAP feature attribution vector that produced it, (2) the Safety Gate pass/fail decision and failure reasons if any, (3) the LLM prompt and response pair, (4) a SHA-256 hash chain linking the recommendation to its source model version. The audit log is append-only and stored in a tamper-evident structure suitable for regulatory inspection.

Fairness metrics. We compute Disparate Impact (DI), Statistical Parity Difference (SPD), and Equal Opportunity Difference (EOD) across protected attributes (age group, gender, income quintile, region type, lifecycle stage) for each task. The fairness monitor runs as a scheduled batch job and generates alerts when any metric exceeds configurable thresholds.

8 Discussion

8.1 Findings Summary

Five findings emerge from the end-to-end pipeline evaluation.

Finding 1: Tree-based distillation works without temperature scaling. Contrary to the standard Hinton distillation recipe ($T = 3$–20), we observed during development that LGBM students trained with $T = 1$ achieve lower JSD and calibration gap than those trained with $T = 5$ (the detailed T comparison is not reported in the experiments section as it was conducted during hyperparameter selection; the design rationale is described in Section 3.1). This is consistent with the Soft GBM analysis [5]: tree models learn from the absolute values and ordering of soft labels, not from gradient flow through tail probabilities. The temperature hyperparameter, designed for neural student backpropagation, is unnecessary — and harmful — for split-based learners.

Finding 2: LGBM gain-based feature selection preserves explanation quality. Selecting features by cumulative LGBM gain (95% threshold, 40–80 features per task) retains the features that the serving model actually uses, ensuring SHAP attributions at serving time remain meaningful. Features like the HMM lifecycle growth-probability feature and the synthetic monthly-spend aggregate carry high gain for lifecycle and engagement tasks respectively, and also provide the narrative anchors ("growth stage," "spending pattern") that ground LLM-generated explanations.

Finding 3: The Safety Gate is essential, not optional. Template-based fallback without LLM validation produces structurally grounded but sometimes less fluent reasons (e.g., Korean particle artifacts like "(를)" as shown in Table 13) and occasionally misleading content (e.g., citing features not actually influential for the customer). The Safety Gate catches these by cross-referencing generated text against the actual LGBM SHAP attribution vector, reducing hallucination-like errors.

Finding 4: Adaptive distillation routing as MRM safeguard. The teacher threshold gate prevented 6 tasks from receiving low-quality distillation: 3 were redirected to direct hard-label training (DIRECT) and 3 to the rule engine (SKIP). Two multiclass tasks (nba_primary 7-class, segment_prediction 4-class) and one regression task (mcc_diversity_trend) were routed to DIRECT; one multiclass task (next_mcc 50-class) and two regression tasks (product_stability, cross_sell_count) fell below floor and were routed to SKIP (Layer 3). This automatic quality triage aligns with SR 11-7 principles: model outputs are monitored, and degraded components are isolated without disrupting service. The monitoring dashboard surfaces these tasks as requiring teacher improvement at the next retraining cycle, creating a closed feedback loop between distillation quality and teacher development priorities.

Finding 5: Per-prediction causal audit pair closes the Art. 13 / GDPR Art. 22 gap. Model-promotion

records track **which** model is in production but do not answer the adjacent per-decision questions **why did the model recommend this** and **can we trust this recommendation**. Combining the Causal Explainability Head attribution (the companion loss-dynamics paper's causal-attribution finding) with the Causal Guardrail coherence score (its causal-guardrail findings) and routing both through the same HMAC-signed hash-chained `AuditLogger` produces a per-prediction record that reconstructs both questions forensically. Three layers of evidence per entry — top-K feature summary for human reading, SHA256 hash of the full attribution vector for cryptographic replay, and hash-chain linkage for tamper detection — let an auditor distinguish authentic records, altered records, and selectively deleted records without trusting the storage medium. The integration required no model surgery; both primitives were existing by-products of the causal expert's forward pass.

8.2 The Dual Role of Features

A key insight from this work: features serve dual purposes in financial recommendation. Even features with marginal predictive contribution (e.g., TDA topological features may add only $\Delta\text{AUC} = 0.01$) provide irreplaceable context for recommendation reasoning. Internally, TDA persistence captures behavioral shape stability — but the customer never sees "persistent homology" or "Betti numbers." Instead, the interpretation registry reverse-maps this to business language (e.g., "You maintain a stable transaction pattern"), and the LLM agent weaves it into a natural-language reason.

This reframes feature engineering evaluation: the value of a feature is not solely its predictive contribution but also its contribution to the recommendation context available to the reason generation pipeline. As argued in the companion paper, *what to observe* matters more than *how to model* — features derived from domain-specific questions ("Is their income permanent or transitory?", "Is product adoption spreading like contagion?") enrich the internal reasoning context, enabling more nuanced business-language explanations than any amount of architectural sophistication applied to shallow statistical summaries.

8.3 Dual Causal Pathways: Designed vs. Discovered

The architecture provides two complementary mechanisms for encoding causal relationships between tasks:

Logit transfer (described in the companion paper, Section 3.7) encodes *known* causal pathways that domain experts design explicitly. For example, churn signal → product stability (customers who churn show declining product engagement) and next MCC category → NBA primary (consumption patterns predict product affinity). These are *codified domain knowledge* — the system encodes relationships that practitioners already understand.

The Causal expert (NOTEARS DAG) (companion paper, Section 3.4) discovers *unknown* causal pathways from data. By learning a directed acyclic graph over the feature space, it can identify relationships that practitioners have not anticipated — for instance, that a specific channel usage pattern causes increased overseas transactions, which in turn predicts interest in travel-oriented check card products.

This dual structure has a concrete regulatory advantage: when a regulator asks "why did the model recommend this product to this customer?", the answer can cite both *designed pathways* (logit transfer: "we encoded the known relationship between churn risk and product stability") and *discovered pathways* (Causal expert: "the model identified a data-driven relationship between this customer's channel behavior and investment interest").

Furthermore, the Causal expert's discovered DAG can inform future logit transfer design — if the DAG consistently identifies an A→B pathway across retraining cycles, practitioners can promote it to an explicit logit transfer in the next model version. This creates a *feedback loop* where data-driven discovery gradually enriches the designed causal structure.

Note: in the companion paper's synthetic data ablation, the Causal expert showed negative transfer on the segment classification task, attributed to input routing overlap with DeepFM (both receive the full feature vector) rather than a fundamental limitation of causal discovery. Validation on production data with genuine causal structure is required to assess the expert's contribution to both predictive performance and explanation quality.

8.4 Practical Deployment Considerations

- **LLM selection**: On-premises LLM vs API (latency, cost, data residency).
- **Reason quality maintenance**: Periodic human review + automated quality scoring.
- **Regulatory updates**: Architecture supports adding new compliance checks without redesign.

8.5 Fidelity Gate Semantics under Cross-Architecture Distillation

The v14 SageMaker distillation re-run (2026-05-08, `m5.4xlarge`, 3-layer fallback enabled) surfaced two latent defects in the fidelity gate that were masked on v13

phase0 by the teacher's lower aggregate AUC. Both defects are now fixed and the resulting semantics are worth documenting explicitly because they shape how a reviewer should read the gate's pass/fail tallies in the audit trail.

Defect 1: the gate scored fallback-routed tasks against their own fallback rationale. The 3-layer serving fallback (CLAUDE.md §1.8) routes a task to Layer 2 (direct hard-label LGBM) when the teacher's discriminative quality is below the viability threshold, and to Layer 3 (rule engine) when it falls below floor. The previous implementation passed **all** tasks with trained students through `validate_fidelity`, so a task that was explicitly routed away from soft-label distillation **because** the teacher was unreliable would then be scored on "how closely the student matches the teacher" — exactly the divergence the routing was designed to introduce. The fix scopes the gate to the `distill_tasks` set returned by `evaluate_teacher_thresholds` and emits an explicit `[SKIP-GATE]` log line for the fallback-routed tasks so the audit log records **why** a task was excluded rather than appearing as a silent omission. The post-fix `fidelity_report.json` carries a new `gate_scope` field (`"distilled_tasks_only"` or `"all_tasks_with_students"`) and a `scoped_tasks` list, so a regulator can verify which tasks fell under which control without inspecting the source.

Defect 2: `calibration_gap` confounded teacher and student calibration. The historical definition, $|\text{ECE(teacher)} - \text{ECE(student)}|$, treats a perfectly Platt-scaled student against a focal-loss-trained teacher (whose calibration is by-construction poor because focal optimises ranking-not-calibration on class-imbalanced binary tasks) as a **failure**: the student's near-zero ECE differences against a 0.25 teacher ECE produces a 0.25 gap, dwarfing the audit threshold 0.05. Yet from an operational standpoint the relevant question is "is the student's probability value trustworthy at a decision threshold?" — i.e., the student's **own** ECE against ground-truth labels. The fix redefines `_compute_calibration_gap` to return student ECE directly; teacher predictions are no longer consulted. The same 0.05 threshold now interprets as an absolute calibration quality requirement on the production scorer rather than a similarity requirement that the teacher's defects rule out.

Residual signal: agreement and ranking-correlation gaps are architecture-inherent, not student defects. After both fixes, the v14 binary distillation set still reports `min_agreement` ≈ 0.75–0.82 and `min_ranking_corr` ≈ 0.87–0.91 against thresholds 0.85 and 0.90. These metrics measure the **local** match between teacher and student decisions (top-1 argmax agreement, Spearman correlation of scores), and their inherent floor depends on the teacher-student architecture pair. Same-architecture distillation (e.g., BERT teacher → BERT student, GPT teacher → GPT student) routinely achieves agreement > 0.92 because the student inherits the teacher's inductive bias; the heterogeneous expert PLE → LGBM pair in this work is **cross-architecture**, with a smooth multi-layer mixture-of-experts teacher and a tree-ensemble student that expresses decision boundaries as axis-aligned step functions. The student can match the teacher's **global** ranking (witness the 0.0058 mean AUC gap, well below the abstract's claim of < 3.6 pp) while disagreeing on individual point estimates exactly because LGBM cannot interpolate a smooth decision surface.

Defect 3: a single-tier gate conflates operational defects with architecture floors. The v14 v4 run after fixes 1 and 2 still reported $\frac{0}{7}$ Tier 1 task pass rate, **not** because the students were unfit for production but because all four conditions (AUC gap, calibration ECE, agreement, ranking correlation) had to clear their thresholds together. Operationally relevant signals (AUC gap, student ECE) cleared their thresholds cleanly on every distilled task; the architecture-inherent metrics (agreement, ranking corr) did not. A gate that always fails forces operator override via `skip_fidelity_gate=true`, which effectively disables the audit control — the same outcome we declined when we refused to relax the thresholds. The fix is to **separate the verdicts** rather than relax either control. The gate now publishes two tiers, distinguished by what the metric measures:

- **Tier 1 (blocking, operational).** Metrics that determine whether the production scorer is broken: auc_gap and student ECE (binary), f_1-macro gap (multiclass), MAE / RMSE gap (regression). Failure here blocks deployment.
- **Tier 2 (informational, diagnostic).** Metrics that measure teacher-student behavioural similarity: top-1 agreement, ranking correlation, Jensen-Shannon divergence (binary + multiclass), quartile agreement (regression). Violations are recorded in `fidelity_report.json`'s `informational_failures` field and aggregated as `tier2_violation_count`. They do **not** affect the report's `status` or the gate's pass/fail tally.

Under this split, the seven distilled tasks of the v14 v4 run report Tier 1 $= \frac{6}{7}$ pass and Tier 2 $= \frac{7}{7}$ violations (architecture floor); the single Tier 1 residual is `cross_sell_count`'s `rmse_gap`, whose semantics

are analysed in the companion loss-dynamics paper's Finding 14. The split makes the operational verdict unambiguous while still surfacing the cross-architecture signal for monitoring. A regulator reviewing the audit log can therefore distinguish "is this student broken?" (Tier 1, today's verdict: no for six of the seven tasks, with the `cross_sell_count` regression gap as the one open item) from "how similar is this student to the teacher?" (Tier 2, today's verdict: as expected for the PLE → LGBM pair). Future cross-architecture runs report their own Tier 2 trajectory; a sudden agreement collapse (e.g., to < 0.5) still surfaces as an anomaly via the `tier2_violation_count` trend, even though it never had the power to block on its own. A separate audit metric for **cross- architecture** runs (Tier 2 **delta vs. the historical floor**) is a natural follow-up but not adopted here.

8.6 Limitations

- LLM dependency introduces cost, latency, and residual hallucination risk.
- Human evaluation scale limited by expert availability.
- Korean/EU regulatory focus; other jurisdictions not yet analyzed.
- Template fallback quality is inherently limited.
- **Per-prediction causal audit runs on the PLE teacher, not the LGBM student.** The serving path at `core/serving/predict.py` uses distilled LGBM models; CEH attribution and CG coherence live on the teacher's causal expert. A periodic teacher-path inference or a parallel teacher monitor is therefore required to emit per-prediction audit events for production traffic. A direct teacher-in-serving path is future work.
- **Synthetic OOD probes only for CG.** The Causal Guardrail was validated on three synthetic out-of-distribution probes (uniform random, column-permuted, extreme-tail). Real-world distribution drift (temporal shift, subgroup imbalance, adversarial) is expected to differ in structure and is not yet evaluated.
- **Attribution meaningfulness not human-evaluated.** CEH v2 produces per-sample attributions that discriminate across samples (the companion loss-dynamics paper's post-hoc attribution evaluation), but whether the top-K features align with domain-expert expectations or with alternative attribution methods (Integrated Gradients, DAG-path traversal) has not been assessed. A human-evaluation pass is planned.

8.7 Future Work

- Real customer A/B testing of reason quality → conversion rate.
- Multi-lingual reason generation (Korean, English, Chinese).
- Automated regulatory update pipeline (regulation change → compliance check update).
- Fine-tuned domain-specific small LLM to replace general-purpose API.
- **Auditor query UI** over the HMAC-signed audit store, with regulator- facing views for attribution records, guardrail triggers, and model-promotion history under a unified query interface.
- **Teacher-in-serving monitor** that runs the PLE teacher in parallel with the LGBM student and emits per-prediction `log_attribution`
  1. `log_guardrail` events, closing the gap identified in the

  limitation above.
- **Extension to remaining Axis-3 candidates** (CTGR / CRCG / CCP from the companion loss-dynamics paper) once their MV work lands, plus a primary-task- gradient CEH variant that aligns attribution with specific task logits rather than the causal-encoder's aggregate output.

8.8 Ethics and Data Statement

All experiments in this version use **synthetic benchmark data** (1M customers generated via Gaussian Copula + latent variable variance budget with fixed seed). No real customer data is included in this version. Validation on production data is planned for a subsequent revision. The production system design targets low-risk check card products only; investment and insurance product recommendations are explicitly excluded from the deployment scope (Section 6.3.1). The system is designed to comply with Korean FSS AI guidelines, the EU AI Act, and the Korean AI Basic Act, with automated fairness monitoring across 5 protected attributes.

9 Conclusion

We presented a full-chain system that bridges the gap between model prediction and human persuasion in financial product recommendation.

Five key contributions define this work:

1. **Adaptive knowledge distillation with three-layer fallback**: teacher threshold gating routes each of 13 tasks to DISTILL (7), DIRECT (3), or SKIP (3) based on teacher quality, ensuring service continuity per SR 11-7 model risk management. LGBM gain-based feature selection aligns with the serving model perspective, replacing OOM-prone teacher IG attribution.

2. **3-agent recommendation reason generation pipeline** (Feature Selector → Reason Generator → Safety Gate) produces natural-language explanations grounded in business-mapped feature attributions, with role separation enabling independent improvement and audit logging.

3. **Two operational agents** (OpsAgent and AuditAgent) interpret monitoring and compliance outputs in natural language, eliminating dashboard fatigue and enabling regulation-compliant MLOps for small teams without dedicated MLOps staff — extending the architecture to a 5-agent system (3 serving + 2 ops).

4. **Regulatory compliance embedded by design** — Korean FSS guidelines, the EU AI Act, and the Korean AI Basic Act are explicitly mapped to system architecture components, with automated monitoring (drift, fairness, herding) and human-in-the-loop oversight at critical decision points.

5. **Per-prediction causal audit pair** — the Causal Explainability Head attribution (the companion loss-dynamics paper's causal-attribution finding) and the Causal Guardrail coherence score (its causal-guardrail findings) flow into the same HMAC-signed hash-chained audit store via `log_attribution` and `log_guardrail`, producing a per-decision record that pairs **what** the model recommended with **whether** that recommendation can be trusted. Satisfies GDPR Art. 22 meaningful- explanation and EU AI Act Art. 13 transparency obligations with cryptographic tamper-evidence.

The fundamental insight is that features serve a dual role in financial AI: they contribute to prediction *and* to the explanation vocabulary that ultimately persuades customers, empowers relationship managers, and satisfies regulators. This reframes the traditional feature engineering calculus: a feature with marginal AUC contribution but rich business interpretability may be more valuable than a high-AUC feature that generates no meaningful explanation.

The system is designed for deployment on serverless infrastructure (AWS Lambda), achieving 120ms warm latency (L1 predict + 13 tasks) without dedicated GPU servers — with L2a on-demand Bedrock rewrite at first call (2.4s, cached at 6ms thereafter) and cold start 6s (S3 model download), matching the operational reality of financial institutions with limited ML engineering resources.

Author Contributions

Seonkyu Jeong (PM / Lead Architect / Data Scientist): Designed the distillation strategy, recommendation reason generation pipeline, and regulatory compliance architecture. Led the overall system design based on domain expertise in financial risk management.

Euncheol Sim: Feature reverse-mapping registry, vector database pipeline, and data ingestion for the serving layer.

Youngchan Kim: Knowledge distillation implementation, model training validation, and mathematical verification of distillation quality.

All authors collaborated through Scrum sprints with rapid feedback cycles.

Funding

This research received no external funding, grants, or institutional infrastructure support. All costs — including AI tools, hardware, mobile connectivity, AWS SageMaker cloud training (Spot instances), S3 storage, and operational expenses — were borne entirely by the first author's personal funds, with development conducted on a single desktop GPU in a resource-constrained environment.

Acknowledgments

The authors thank Yeon-Jin Kim for consistently providing valuable insights on industry trends and marketing perspectives that informed the system's design direction.

The authors express deep gratitude to Euncheol Sim and Youngchan Kim, who dedicated countless nights and weekends to this project with unwavering commitment despite the absence of formal employment or compensation.

Finally, the authors wish to acknowledge the AI tools that made this research possible. Gemini for ideation and design brainstorming, Claude Opus and Sonnet for architecture, implementation, and writing, and Cursor for a seamless development environment. What could have remained an unrealized vision was brought to life through the collaboration of a small team and these tools.

Appendix A. Implementation Reference

The concepts described in this paper have concrete Python implementations available in the public code repository.[2] The mapping between paper concepts and implementation modules is as follows:

Paper Concept	Implementation Module
Interpretation registry	`core/recommendation/reason/interpretation_registry.py`
Reverse-mapping layer	`core/recommendation/reason/reverse_mapper.py`
Fact extraction layer	`core/recommendation/reason/fact_extractor.py`
3-agent serving pipeline	`core/recommendation/reason/async_orchestrator.py`
Self-check / Safety Gate	`core/recommendation/reason/self_checker.py`
Template engine	`core/recommendation/reason/template_engine.py`
Operations agent	`core/agent/ops/`
Audit agent	`core/agent/audit/`
Consensus mechanism	`core/agent/consensus.py`
Diagnostic case knowledge base	`core/agent/case_store.py`
Temporal fact store	`core/agent/temporal_fact_store.py`
Dialog recall memory	`core/agent/dialog_recall.py`
Tool registry	`core/agent/tool_registry.py`
Configuration files	`configs/financial/*.yaml`

Table 16: Mapping of paper concepts to implementation modules.

Detailed implementation, unit tests, and configuration files are available in the public repository.

10 Appendix

10.1 Appendix A: Layer 3 Rule-based Fallback per Task

When both teacher distillation (Layer 1) and direct LGBM training (Layer 2) are unavailable, the following task-level rules activate. Each rule is grounded in established marketing theory or financial domain practice, organized by Financial DNA task group.

All 13 rules share a common regulatory floor: the customer's risk tolerance grade must equal or exceed the recommended product's risk grade (Financial Consumer Protection Act, Article 17). Detailed rule specifications, thresholds, and recommendation reason templates are maintained in the design document (`docs/design/12_rule_based_fallback.md`).

Layer 3 rules leverage pre-computed features from Phase 0 (11 academic disciplines including TDA, HGCN, Mamba, GMM clustering, causal discovery, and economics). These engineered features enable rule-based recommendations that are substantially more sophisticated than raw RFM heuristics, at zero additional inference cost.

Bibliography

[1] Board of Governors of the Federal Reserve System. 2011. *Supervisory Guidance on Model Risk Manage-*

[2]https://github.com/bluethestyle/aws_ple_for_financial

DNA Group	Task	Rule	Theory	Trigger	Key Features
Engagement	churn_signal	RFM 30/60/90-day decline	Relationship Marketing (Berry '83)	R/F/M all declining	HMM lifecycle, TDA persistence, Mamba temporal
Engagement	top_mcc_shift	MCC entropy change > threshold	McKinsey CDJ triggers	Lifestyle shift detected	TDA local, merchant_hierarchy, Mamba temporal
Lifecycle	nba_primary	Product adjacency +1 step	Kotler 5A journey	Gap in product ladder	GMM cluster, LightGCN, economics PIH
Lifecycle	segment_prediction	Balance x Frequency x Products	CLV tiered model (Pareto)	Segment re-classification	GMM cluster ID, HMM behavior, TDA global
Lifecycle	will_acquire_deposits	Surplus ratio > 30% + no term deposit	Lifecycle savings stage	Idle cash detected	economics PIH, HMM journey, GMM cluster
Lifecycle	will_acquire_investments	Suitability grade >= product risk	Suitability (KFCPA Art.17)	Risk-matched opportunity	causal NOTEARS, HGCN hyperbolic, economics
Lifecycle	will_acquire_accounts	Salary pattern + non-primary	SOW expansion (PwC)	Primary bank conversion	LightGCN, Mamba temporal, txn_behavior
Lifecycle	will_acquire_lending	Credit grade 1–4 + DTI < 40%	Credit scoring + suitability	Refinance opportunity	causal NOTEARS, economics PIH, HMM lifecycle
Lifecycle	will_acquire_payments	Top MCC + single card holder	Habitual buying (Kotler)	Spending-card mismatch	merchant_hierarchy, TDA local, Mamba temporal
Value	product_stability	30/60/90-day dormancy EWS	Customer engagement (Gallup)	Activity decline	Mamba temporal, HMM behavior, TDA persistence
Value	cross_sell_count	CLV tier target - current holdings	Share of wallet (PwC)	Product gap > 0	LightGCN, GMM cluster, HGCN
Consumption	next_mcc	MCC frequency Top-K + seasonality	Habitual buying behavior	Pattern continuation	Mamba temporal, merchant_hierarchy, TDA local
Consumption	mcc_diversity_trend	PIH transitory income signal	Friedman PIH (1957)	Income shock detected	economics PIH, TDA global, GMM

Table 17: Layer 3 rule-based fallback rules per task. All rules subject to KFCPA Article 17 suitability constraint.

ment. Retrieved from https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm

[2] European Banking Authority. 2025. Report on Machine Learning in Internal Models.

[3] European Parliament and Council. 2024. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act).

[4] Peter S. Fader, Bruce G.S. Hardie, and Ka Lok Lee. 2005. RFM and CLV: Using Iso-Value Curves for Customer Base Analysis. *Journal of Marketing Research* 42, 4 (2005), 415–430.

[5] Jiawei Feng, Yang Yu, and Zhi-Hua Zhou. 2020. Soft Gradient Boosting Machine. In *arXiv preprint arXiv:2006.04059*, 2020.

[6] Financial Services Commission (Korea). 2024. Financial Sector AI Guidelines.

[7] Milton Friedman. 1957. *A Theory of the Consumption Function*. Princeton University Press.

[8] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015.

[9] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems* 20, 4 (2002), 422–446.

[10] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[11] Alex Kendall, Yarin Gal, and Roberto Cipolla. 2018. Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[12] Scott M Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] National Assembly of Korea. 2024. Act on the Development of Artificial Intelligence and Establishment of Trust (AI Basic Act).

[14] Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.

[15] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. Why Should I Trust You?: Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[16] Amr Salih and others. 2023. Examining the Limitations of Post-hoc Explainability Methods in Finance. *Frontiers in Artificial Intelligence* (2023).